2025 K-Software Empowerment Bootcamp

# AI 기반 작물 질병 이미지 분석을 통한 도시농업 지원 시스템(“텃밭닥터”)

**팀명: 파밍패밀리**

**2025.08.17**

| | |
|---|---|
| 프로젝트 분야 | 인공지능(AI) |
| 소스코드 URL | https://github.com/KSEB-AI-3/gardendoctor-app<br>https://github.com/KSEB-AI-3/gardendoctor-backend<br>https://github.com/KSEB-AI-3/gardendoctor_ai-server |
| 시연영상 URL | |

# 목차


**1. 개요** ..... 3

1.1 프로젝트 요약 ..... 3

1.2 프로젝트 팀 구성 및 역할 ..... 5

**2. 프로젝트 주차별 보고서** ..... 5

2.1 주차별 프로젝트 작업 현황 요약표 ..... 5

2.2 WBS 및 세부 추진 일정 ..... 6

**3. 프로젝트 결과 보고** ..... **8**

3.1 문제 정의 및 기획 ..... 8

3.2 기술 구현 및 설계 ..... 22

**4. 부록** ..... **57**

4.1 모델 성능 비교표 ..... 58

4.2 코드 스냅샷 ..... 60

4.3 UI/UX 화면 ..... 65

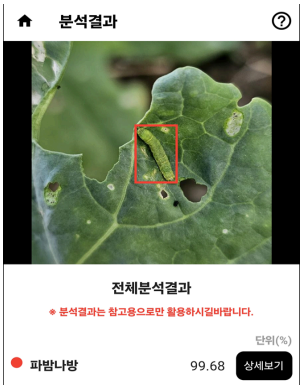
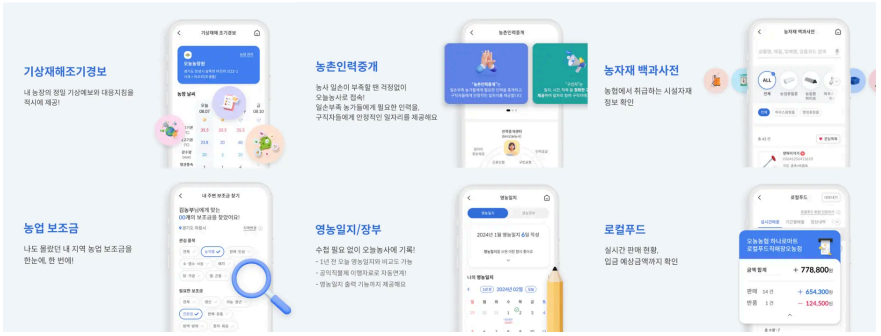
4.4 이슈 관리 ..... 67

4.5 설치/운영 매뉴얼 ..... 60

# 1. 개요

## 1.1 프로젝트 요약

| 구분 | 내용 |
|---|---|
| 프로젝트 명 | 텃밭닥터 – AI 기반 통합 도시농업 지원 애플리케이션 |
| 수행기간 | 2025.07.01 ~ 2025.08.28 (총 8주) |
| 주요목표 | • 도시농업의 지속 가능성을 높이고 농업 참여를 활성화<br>• 물리적 제약(경작 공간 부족)과 비물리적 제약(전문 지식 부족·병해충 대응 한계) 완화를 핵심 과제로 설정<br>• AI 기반 병해충 진단, 24시간 챗봇 상담기능 구현<br>• 나의 작물 일지 작성 및 관리, 위치 기반 농장 정보 제공 |
| 적용분야 | 도시농업·스마트팜, 친환경·지속가능 농업 서비스, 디지털 농업 교육 및 컨설팅 |
| 핵심기능 | 초보자: 작물 재배 경험 부족, 병해충 문제 시 즉시 대응 어려움<br>기존 참여자: 안정적인 재배 환경 부족, 추가 경작 공간 확보 어려움<br>• AI 농업 상담 챗봇: 24시간 실시간 상담, 농촌진흥청 데이터·농사로 API 연동, 자연어 질의응답<br>• 위치 기반 농장 정보 제공: GPS 기반 20km 이내 농장 검색, 분양 가능 여부, 시설 정보 제공<br>• AI 기반 병해충 진단: 스마트폰 사진 업로드로 즉시 진단, 12종 주요 작물 질병 분석, 맞춤형 치료·예방 가이드 제공, 진단 이력 관리<br>• 작물 성장 관리 일지: 파종~수확까지 디지털 일지, 일지별 알림, 사용자 작물 데이터 관리 |

| KPI 핵심 지표 | 내용 |
| --- | --- |
| **AI 병해충 진단 정확도 ≥ 95%** | 농촌진흥청 병해충 영상진단 AI 평균 정확도 95% 수준<br>  <br><br><스마트 병해충 진단 시스템> |
| **작물 피해율 감소율 ≥ 50%** | 농진청 <스마트 병해충 진단 시스템> 작물 **피해율**이 연간 **30~40%**까지 **감소**하는 효과와 **수확량 20~30% 증가** |
| **신규 참여율 증가폭 +40%** | 서울시 공유텃밭, 학교·공동체 텃밭 운영 시 신규 유입률 평균 **18~27%** 상승 |
| **재방문율 ≥ 70%** | NH오늘농사 앱(농협 영농 정보 서비스)<br>월간 재방문율이 50% 이상(최대 62% 수준)을 꾸준히 기록<br>  <br><br><NH오늘농사> |
| **DAU (일일 활성 사용자) 5,000명** | 농진청 병해충 진단 앱, NH오늘농사, 농사로 등 유사 서비스<br>출시 초/평균 DAU가 500~2,500명 수준 |

### 1.2 프로젝트 팀 구성 및 역할

| 이름 | 역할 | 담당업무 | 참여기간 |
|---|---|---|---|
| 박관호 | 팀장 / AI 엔지니어 | 프로젝트 총괄, 이미지 분석 모델 개발, 병해충 진단 모델 서빙 환경 구축 | 8주 |
| 김인희 | 팀원 / 백엔드 | API 서버 개발, 데이터베이스 설계, 외부 API 연동 | 8주 |
| 김용빈 | 팀원 / 백엔드 | API 서버 개발, 데이터베이스 설계, 외부 API 연동 | 8주 |
| 권민우 | 팀원 / 프론트 | 모바일 애플리케이션 UI 개발, 사용자 경험 최적화 | 8주 |
| 남은지 | 팀원 / PM, AI | 프로젝트 기획 및 관리, AI 챗봇 시스템 개발 | 8주 |

## 2. 프로젝트 주차별 보고서

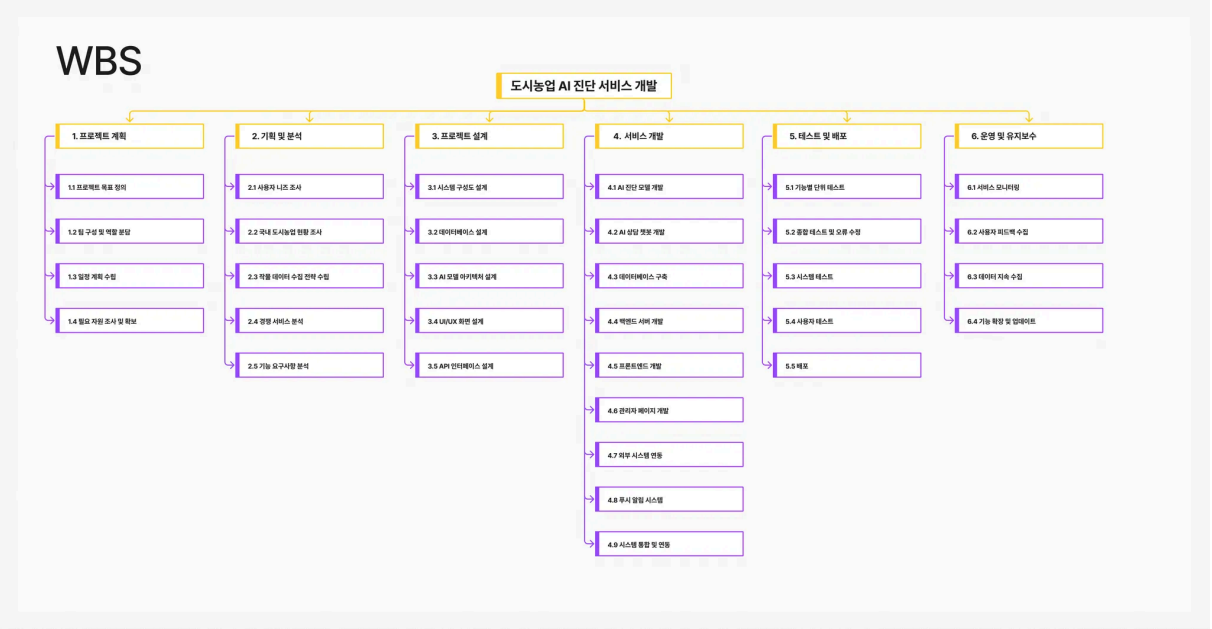
### 2.1 주차별 프로젝트 작업 현황 요약표

| 주차 | 멘토링 | 강사 피드백 | 이슈 대응 | 기술 학습 | 업무 분담 |
|---|---|---|---|---|---|
| **1주차** 7/1 ~ 7/4 | | 전체 아키텍처 구체화, API 설계 시 데이터 표준 준수 권고 | 병해충 데이터 분류 기준 불명확 → 농촌진흥청 분류 체계 재검토 | AIHub·농촌진흥청 데이터 구조 분석, API 명세 작성법 | [공통] 기획·설계 / AI: 데이터셋 조사 / BE·FE: 기술 스택 확정 |
| **2주차** 7/7 ~ 7/11 | 데이터 전처리 및 AI 학습환경 설정 검토 | 이미지 증강 기법·품질 유지 방안 제안 | AI Hub 데이터 일부 품질 불량 → 필터링 로직 추가 | PyTorch 이미지 증강, Flutter 기본 레이아웃 구성 | AI: 전처리·증강 / BE: 서버환경·인증 API / FE: 기본 UI 구조 |
| **3주차** 7/14 ~ 7/18 | CNN 모델 초기 성능 점검, GPT API 활용성 검토 | 모델 정확도 개선 방안, GPT 검색 응답 품질 개선 | 모델 초기 정확도 82% → 데이터 증강 비율 조정 | CNN 구조, LangGraph 기반 에이전트 설계 | AI: 모델 1차 학습 / BE: 식물·농장·일지 API / FE: 사용자 기능 UI |
| **4주차** 7/21 ~ 7/25 | AI 모델 고도화, 챗봇 아키텍처 검토 | 벡터 DB 연동 시 성능 튜닝 포인트 제공 | LangGraph 연동 오류 → API 호출 시 스키마 수정 | FAISS 벡터 검색, 지도 API 연동 | AI: 고도화 / BE: 알림·챗봇 API / FE: 지도·챗봇 UI |

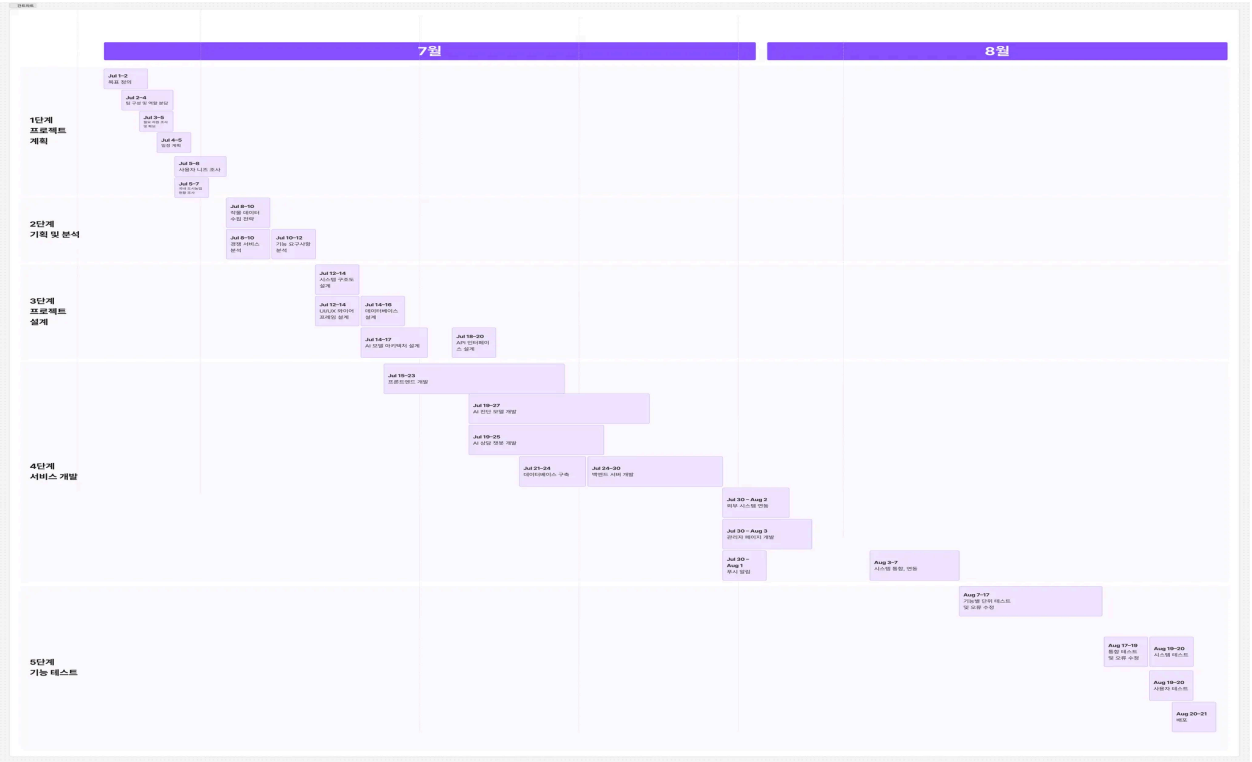
| | | | | | |
|---|---|---|---|---|---|
| **5주차**<br>7/28 ~ 8/1 | 모델 통합 테스트 전략 자문 | API-모델 통합 시 에러 핸들링 구조 강화 피드백 | 이미지 업로드 시 대용량 처리 오류 → 비동기 처리 적용 | API-모델 연동, 반응형 UI 디자인 | AI: 모델 최적화 /<br>BE: API 통합 /<br>FE: 일지 기능·API 연동 |
| **6주차**<br>8/4 ~ 8/8 | 통합 테스트 계획 점검 | 테스트 시나리오의 예외 케이스 추가 권고 | 다중 사용자 동시 접속 부하 문제 → Redis 캐시 적용 | 부하 테스트 기법, AWS 배포 환경 구성 | FE/BE: UI·성능 개선 /<br>AI: 안정성 강화 /<br>전체: 통합 테스트 |
| **7주차**<br>8/11 ~ 8/15 | 베타 테스트 피드백 수렴 | 사용자 시나리오 기반 QA 강화 | UI 반응 속도 저하 → 이미지 최적화 적용 | QA 프로세스, 버그 관리 툴 사용법 | 전체: 버그 수정·최종 최적화 |
| **8주차**<br>8/18 ~ 8/20 | 최종 발표 리허설 지도 | 발표자료 시각화·스토리라인 보강 | AWS 배포 환경 세팅 중 SSL 인증서 오류 해결 | 발표 자료 제작 툴, 기술 문서 표준 | 전체:<br>배포·문서화·발표 준비 |

## 2.2 WBS 및 세부 추진 일정

### 2.2.1 작업 세분화 표



## 2.2.2 간트 차트




WBS_AI 3팀_파밍패밀리.xlsx

## 2.2.3 주차별 세부 진행 내역 작성

### 주간 프로젝트 일지

| 팀 명 | 파밍패밀리 | 일자 | 2025년 7월 4주차 | |
|---|---|---|---|---|
| 주 요 추진업무 | 업 무 내 용 | 완결 | 산 출 물 | 작업자 |
| | -토마토/고추 병해 분류 모델 개발 및학습<br>-기존 모델 성능 개선<br>-토마토/고추 질병 이미지 데이터 전처리 | ■ | 토마토/고추 병해 분류 학습 모델<br>성능 고도화 모델<br>전처리된 토마토/고추이미지데이터셋<br>모델 성능 평가 보고서 | 박관호 |
| | -소셜 로그인/회원가입 백엔드-Flutter 연동 개발<br>-주변 텃밭 위치 지도 API 백엔드-Flutter 연동 개발<br>-챗봇 FastAPI-Flutter 연동 개발 | ■ | 소셜 로그인/회원가입 연동 모듈<br>텃밭 위치 지도 연동 모듈<br>챗봇 API 연동 모듈<br>Flutter 앱 통합 코드<br>API 연동 문서 | 권민우 |
| | -전체 발송 알림 기능 개발<br>-Form-login버그 및 swagger RequestBody import 오류 수정<br>-Image domain 도입으로 인한 Diary/User/Analysis 도메인 리팩토링<br>-예외처리 및 swagger 문서 추가 | ■ | 전체 발송 알림 기능 모듈<br>수정된 로그인 기능<br>리팩토링된 Diary/User/Analysis 도메인 코드<br>예외처리 로직<br>swagger API 문서 | 김용빈 |
| | -식물/사용자식물/텃밭 도메인 리팩토링 및 테스트<br>-Amazon S3 이미지 관리 시스템 구축<br>-이미지 처리 통합 관리 전환<br>-농림축산식품 API 연동<br>-텃밭 데이터 복구 및 카카오지도 API변경 | ■ | 리팩토링된 도메인 코드<br>S3 이미지 관리 모듈<br>농림축산식품 API 연동 모듈<br>카카오 지도 API 모듈<br>복구된 텃밭 데이터 | 김인희 |
| | -기능별 단위테스트 수행<br>-AI 시스템 연동 작업<br>-전문DB 벡터화 및 웹서치기반 챗봇<br>-챗봇 질의응답 기능 구현<br>-주간보고서 및 회의록 작성 | ■ | swagger API 문서<br>AI 연동 모듈<br>벡터화된 전문DB 데이터셋<br>웹서치 연동 챗봇 시스템<br>주간보고서 및 회의록 | 남은지 |
| 차 주 예정업무 | 업 무 내 용 | | 예정일자 및 예상문제점 | 비 고 |
| | 주요 12개 작물 대상 정확도 90% 이상 모델 최적화, 예외 케이스 안정성 강화 | | 2025.7.28-8.01 / 목표 정확도 달성 어려움, 예외 케이스 처리 복잡성 | |
| | 식물등록/일지등록 백엔드-Flutter 연동, UI 추가 개발, AI 분류모델 연동 | | 2025.7.28-8.01 / API 연동 오류, UI 호환성 문제, 모델 응답속도 지연 | |
| | 클라이언트 연동 테스트, DB 쿼리 최적화, n+1 문제 해결, 서버 배포 학습 | | 2025.7.28-8.01 / 성능 병목지점 발견, 배포 환경 설정 이슈 | |
| | 관리자 페이지 개발(타임리프), DB 최적화, 클라이언트 연동 테스트 | | 2025.7.28-8.01 / 타임리프 템플릿 오류, 권한 관리 복잡성 | |
| | 프롬프트 개선, 작물 병해충 진단-챗봇 연결 기능 | | 2025.7.28-8.01 / 프롬프트 성능 최적화 한계, 진단 데이터 연동 오류 | |
| | 산출물/소스코드 정리,<br>요구사항정의서/WBS 작성 | | 2025.7.28-8.01 / 문서 작성 시간 부족, 요구사항 변경사항 반영 | |
| | 전체 시스템 연동 및 안정성 검증 | | 2025.7.28-8.01 / 시스템 통합 시 예상치 못한 오류, 테스트 환경 구축 | |
| 기타사항 | | | | |

#### 📋 주요 논의사항 및 보고사항

**1. 모델 아키텍처 개선 방안**

**현재 상황:**

- 작물별 개별 모델 운영 (12개 작물 × 12개 모델) - 영상시연으로 2-3개로 결정
- 정확도는 높으나 서비스 운영 복잡성 증가

**제안된 개선안:**

- **단일 모델 + 멀티 아웃풋 구조**로 변경
  - Output 1: 작물 분류 (12개 작물 구별)
  - Output 2: 병해충 진단
- 서비스 운영 효율성 향상 vs 정확도 trade-off 검토 필요

**액션 아이템:** 단일 모델 성능 테스트 및 정확도 비교 분석

**2. 진단-챗봇 연동 히스토리 관리**

**현재 문제:**

- 진단 시스템 → 챗봇 전환 시 진단 결과 연결 안됨
- 사용자가 "챗봇에게 물어보기" 클릭 시 컨텍스트 손실

**논의된 해결방안:**

- 진단 히스토리 저장 기간: 최근 3개월 vs 세션 기반
- 데이터 연결 방식: 세션 ID, 사용자 기반 저장 등
- 개인정보보호 고려사항

**액션 아이템:** 히스토리 관리 방식 기술 검토 및 구현 방안 수립

매주 화요일 20시 정기적인 멘토링 진행 , 진행계획과 피드백으로 지속적인 관리

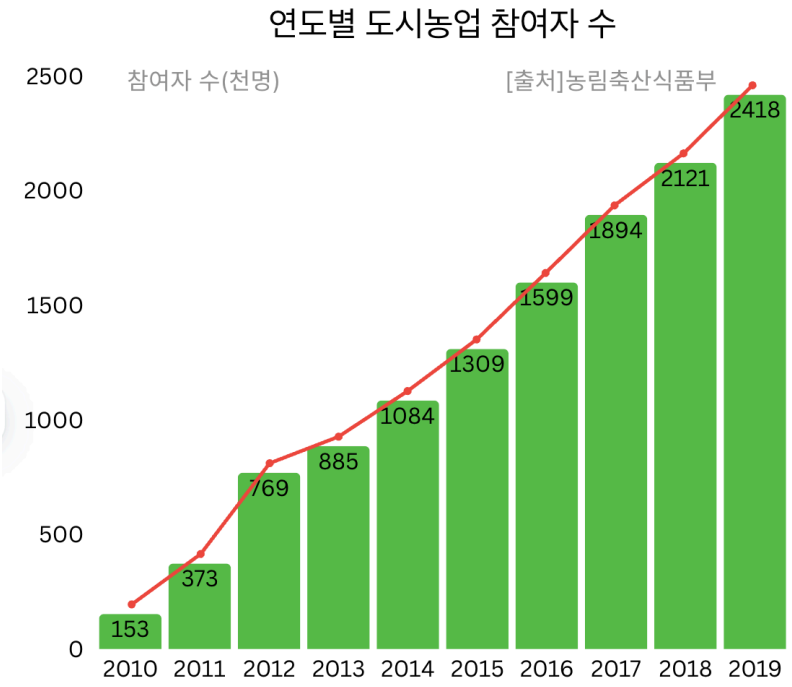
# 3. 프로젝트결과 보고

## 3.1 문제 정의 및 기획

### 3.1.1 배경 및 필요성

#### 1) 도시농업의 급속한 성장과 사회적 관심 증대

코로나19 팬데믹을 계기로 도시농업은 단순한 취미 활동을 넘어 환경 개선, 건강한 여가 활동, 그리고 식품 안전성에 대한 관심이 결합된 새로운 라이프스타일로 자리매김하였다. 농촌진흥청 통계에 따르면, **도시농업 참여자 수**는 2010년 15만 명에서 2019년 241만 명으로 **약 16배 증가**하는 놀라운 성장세를 보였다. 이는 연평균 35% 이상의 성장률에 해당하는 수치로, 도시농업이 사회의 주요 트렌드로 자리잡았음을 보여준다.




#### 2) 확산되는 도시농업, 높아지는 이탈율

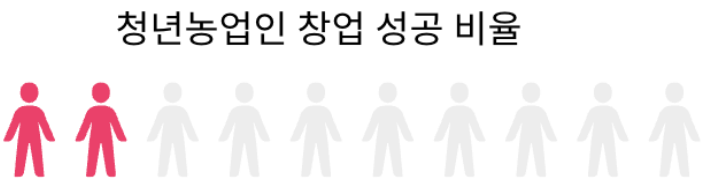
심층K

**홍보는 열심, 관리는 뒷짐…“다시 도시로” [청년농부 절망보고서]④**

입력 2024.04.18 (07:02) | 수정 2024.04.19 (10:49)

*“지난 2월 경북 의성에서 20대 청년이 의식을 잃은 채 발견됐습니다. 스스로 목숨을 끊으려 했던 겁니다. 청년은 결국 3월 초 세상을 떠났습니다. 그가 SNS에 남긴 유서에는 '청년 귀촌의 이면'을 담았습니다.” [출처: 청년농부 절망보고서]*

도시농업 이탈 문제의 심각성은 농촌 귀농 사례에서도 명확하게 드러난다. KBS 보도에 따르면, 전국 청년 귀농·귀촌 1번지로 꼽히는 경북 의성군에서조차 농업 정착에 실패하는 사례가 빈발하고 있다. 2019년부터 2023년까지 의성군 스마트팜 조성 사업에 참여한 청년 농업인 137명 중 **69명(50.4%)**이 교육조차 완료하지 못하고 포기했으며, 그나마 창업한 경우도 **10명 중 2명 수준**(22%)에 불과했다.




더욱 주목할 점은 농촌진흥청과 한국농촌경제연구원이 2014년부터 5년간 귀농인 1,039명을 장기 추적한 결과, 역귀농률이 약 8.6%에 달했다는 사실이다. 즉, 귀농인 10명 중 1명은 농업을 포기하고 도시로 되돌아갔으며, 이들이 역귀농을 택한 주요 요인은 '**영농 실패**'로 나타났다. 실제로 통계청 기준 2012년 57,000여 명이었던 대구·경북 청년 농가는 10년 사이 25,000여 명으로 **절반 넘게** 줄어들었다.

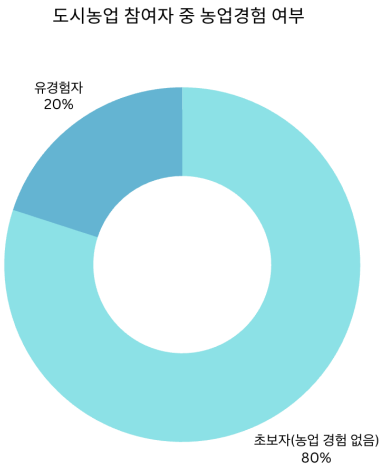
이러한 현상은 도시농업에서도 유사하게 나타나고 있다. 정부와 지자체의 적극적인 홍보와 지원에도 불구하고, 초보 농업인들이 병해충 대응과 작물 관리 기술 부족으로 인해 농업 자체에 대한 신뢰를 잃고 이탈하는 구조적 문제가 존재한다. 의성군 사업 포기 청년의 증언처럼 "저희가 초보 농부이다 보니까 소득이 안 나면 그다음 바로 갚아나가는 것도 힘든" 상황은, 규모의 차이는 있지만 도시농업 참여자들이 겪는 좌절감과 본질적으로 다르지 않다. 이는 단순한 홍보나 공간 제공을 넘어서, **체계적인 기술 지원**과 **전문 상담 시스템**이 농업 정착률 향상의 **핵심 요소**임을 시사한다.

초보 도시농부는 병해충 대응 미흡으로 인해 재배 작물을 잃게 되고, 이는 결국 도시농업 참여의 중단과 이탈로 연결된다. 따라서 **병해충 관리 능력 강화**와 **전문적 지원 체계**의 구축은 도시농업을 장기적으로 유지·발전시키기 위한 **핵심 과제**로 볼 수 있다.

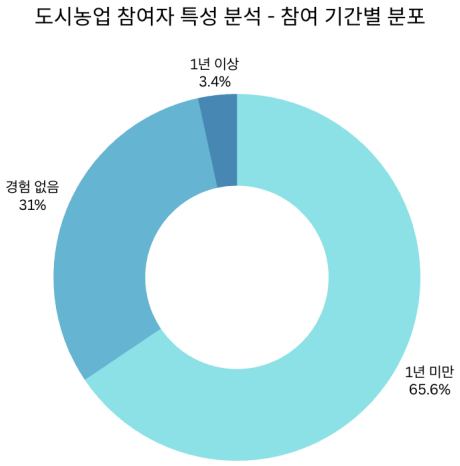
### 3.1.2 현황 및 문제점

#### 1) 병해충 문제와 경험 부족이 초래하는 도시농업 이탈

도시농업 참여자들의 상당수가 초보자로 **1년 미만**의 짧은 경험을 가지고 있으며, 서울 시민 중 약 **83.9%**가 무경험자에 해당한다. 참여 기간별 분포를 보면 1년 미만이 65.5%, 경험 없음이 31.0%로 총 96.5%가 1년 미만 또는 무경험자에 해당했다.

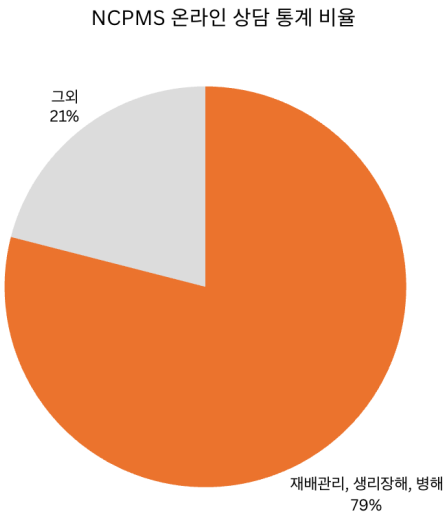



「도시농업 시민인식 조사 결과 보고서」




「도시농업 프로그램이 가족의 유대감과 친밀감 형성 및 도시농업에 대한 인식」

이러한 상황 속에서 도시농업 확산과 함께 병해충 및 생리장해 관련 민원이 급증하고 있다. 경기도농업기술원이 운영한 '사이버 식물병원' 3개년 분석(2011~2013)에 따르면, 온라인·오프라인 진단 의뢰 901건 중 채소가 47.1%, 화훼가 24.3%를 차지했으며, 의뢰 사유는 **생리장해 31.7%, 병해 28.6% 순**이었다. 연구진은 "도시농업인의 급증에도 불구하고 재배 경험 부족이 주요 원인"임을 명시하였다. 또한 농사로 전문가 칼럼(2021)에서는 NCPMS 온라인 상담 통계를 인용해 "**병해충 온라인 의뢰의 79%가 재배관리, 생리장해, 병해 등 초보 농가의 진단 요청**"이라고 보고하였다.




[출처] 농사로 전문가 칼럼(2021)

이러한 문제는 실제 재배 지속률에도 부정적 영향을 미친다. 서울시 '자치구 도시농업지원센터 운영점검'(2021)에서는 텃밭 이용자 중 장기간 병해·생리장해로 수확이 어려워 중도 포기하거나 보조금 환수 사례가 발생한 것으로 보고했다. 한국농촌경제연구원(KREI)의 도시농업 비전 보고서(2010, 2022) 또한 "**도시농업 이탈 원인**의 상당 부분이 **병해충·재배 실패에 기인한다**"고 분석하며, 체계적 교육 및 전문 상담 인프라 부족을 해결해야 할 핵심 과제로 제시하였다.

이로 인해 초보자들의 작물 재배 실패 사례가 지속적으로 보고되고 있어, 초보자들이 안정적으로 도시농업에 정착할 수 있도록 보다 체계적이고 실질적인 가이드 제공이 필요하다.

### 2) 전문 지식 부족과 상담 체계의 한계

**도시농업 참여자의 상당수는 재배 경험이 부족한 초보자 또는 비전문가**로, 작물 관리와 병해충 대응 과정에서 지속적으로 어려움을 겪고 있다. 농민신문 보도에 따르면 발생 현황조차 집계되지 않는 병해충이 다수 존재하며, 이를 **신속·정확하게 식별하고 대응할 수 있는 전문인력은 턱없이 부족한 실정**이다.

기후 변화와 외래 병해충의 확산으로 새로운 병해충이 빈번히 발생하고 있으나**, 전문 인력 부족으로 인해 현장 대응이 지연**되고 있다. 이러한 환경은 초보 도시농부들이 작물 피해 상황에서 적시에 상담을 받기 어려운 구조를 만들며, 이는 곧 **농업 참여 의욕 저하**로 이어지고 있다.

식물병해충 전공 교수들 또한 농업기술원·농업기술센터에 **전담 전문가가 거의 없으며**, 농촌진흥청과 농림축산검역본부에도 **소수**만 근무하고 있다고 지적한다. 이로 인해 병해충 긴급행동지침(SOP)에 작물 특유의 생태적·기술적 특성이 제대로 반영되지 못하는 구조적 문제가 발생한다. 실제로 SOP 작성이 전문가가 아닌 일반 행정공무원 주도로 이뤄지며, 이는 가축질병 분야보다도 전문가 의견이 덜 반영되는 상황을 초래하고 있다.

[식물병해충의 습격] 해마다 피해 반복되는데…통계 없고 전문가 턱없이 모자라

입력 : 2021-07-26 00:00 | 수정 : 2021-07-25 12:33

서울시 농업기술센터 공무직 채용공고에서는 종합분석실 업무(토양,병해충,미생물 등)지원 **채용 인원이 1명**으로, 서울시 이외 지역(세종,예산 등)에서도 병해충 전문가를 채용하고 있지 않으며 부수적인 업무로 처리중이다. 더불어, 농업기술센터 전체 직원 대비 병해충 전문가 비율은 **약 2.8% 미만**이며, 순수 병해충 전문가는 전국적으로 **수십 명** 수준에 불과하다. 즉, 전체 농업기술센터에서 병해충 업무를 담당할 인력이 **극소수**에 불과한 현실을 보여준다.

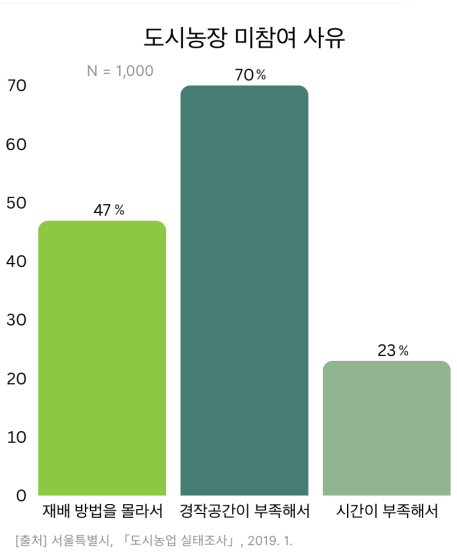
농업기술센터 전체 직원 대비 병해충 전문가 비율

<서울시 농업기술센터 공무직 채용공고>

농촌진흥청 농업기술 상담예약 시스템에서 전화상담은 접수일 기준으로 2일 후부터 예약 가능하며, 현장 방문은 5일 후부터 예약 가능한 탓에, **도시농업 참여자는 상담체계에 어려움을 겪고 있다.**

### 3) 높은 관심 대비 실질적 참여 장벽 존재

더불어, 도시농업에 대한 높은 사회적 관심에도 불구하고, 실제 참여를 가로막는 다양한 구조적 문제들이 존재한다. 농촌진흥청이 실시한 '도시농업 실태 조사'에 따르면, 도시 내 농작물 재배가 어려운 주요 이유로 다음과 같은 결과가 나타났다.
**경작공간 부족이 70.3%, 재배 방법을 몰라서가 46.9%, 시간 부족이 23.4%**로 조사되었다. 이러한 통계는 도시농업 참여 의향이 높음에도 불구하고 실질적인 참여로 이어지지 못하는 현실적 한계를 명확히 보여준다.




<도시농장 미참여 사유>

결론적으로, 위와 같은 구조적 문제들을 해결하기 위해서는 기존의 전통적인 상담 방식을 넘어선 혁신적인 접근이 필요하다. **AI 기반 진단 및 맞춤형 관리 기술**, **24시간 실시간 상담 시스템, 위치 기반 경작 공간 탐색 기능**을 통해 전문성 부족과 기존 농업기술센터의 시간적 제약 문제를 동시에 해결할 수 있는 **통합적 솔루션**이 요구되는 시점이다.

### 3.1.3 문제 해결 방향 및 전략

#### 1) 핵심 전략

텃밭닥터 프로젝트의 핵심 전략은 '기술 기반의 통합 솔루션 제공을 통한 도시농업 진입 장벽 해소 및 참여자 경험의 혁신' 이다. 도시농업의 지속 가능성을 저해하는 물리적, 비물리적 제약을 극복하기 위해, 본 프로젝트는 다음과 같은 다각적인 접근 전략을 설정한다.

**가) 비물리적 제약(지식 및 경험 부족) 해소를 위한 AI 기반 솔루션 전략**

- **AI 기반 병해충 진단**: CNN(Convolutional Neural Network) 기술을 활용한 AI 진단 기능은 스마트폰으로 작물의 사진을 찍는 간단한 행위만으로 즉각적인 해결책을 제시한다. 이는 지식 부족으로 인한 초기 대응 실패를 막고, 작물을 건강하게 키울 수 있다는 자신감을 부여하는 핵심 기능이다.

- **24시간 AI 농업 상담 챗봇**: 진단에서 끝나는 것이 아니라, 후속 조치와 예방, 재배 전반에 걸친 궁금증을 24시간 언제든지 해결할 수 있는 **소통 창구**를 마련한다. 농촌진흥청의 신뢰도 높은 데이터를 기반으로 하여, 사용자는 시간과 장소에 구애받지 않고 **개인 맞춤형 농업 컨설팅**을 받을 수 있다.

- **데이터 기반 작물 성장 관리 일지**: 파종부터 수확까지의 전 과정을 디지털로 **기록**하고 **관리**하게 함으로써, 사용자는 자신만의 농사 데이터를 축적하게 된다. 이는 단순한 기록을 넘어, 성공과 실패의 경험을 **체계화**하여 다음 농사를 위한 귀중한 자산으로 활용하게 만드는 전략적 도구이다.

**나) 물리적 제약(경작 공간 부족) 극복을 위한 연결 전략**

- **위치 기반 농장 정보 제공**: GPS 기술을 활용하여 사용자의 현재 위치로부터 20km 이내에 있는 주말농장, 공동체 텃밭 등의 정보를 제공한다. 분양 가능 여부, 시설 현황 등 구체적인 정보를 투명하게 제공함으로써, 경작 공간을 찾는 데 드는 시간과 노력을 획기적으로 줄여 신규 참여를 촉진한다.

이처럼 텃밭닥터는 AI 진단, 챗봇 상담, 성장 일지, 농장 정보 제공이라는 4가지 핵심 기능을 유기적으로 통합하여 제공한다. 이는 각 기능이 독립적으로 작동하는 것을 넘어,
***'발견(농장 찾기) → 실행(파종 및 기록) → 위기 대응(병해충 진단) → 학습(챗봇 상담)'***
으로 이어지는 도시농업의 전 과정을 끊김 없이 지원하는 하나의 완성된 생태계를 구축하는 것을 최종 목표로 한다. 이를 통해 도시농업의 문턱을 낮추고 누구나 쉽고 즐겁게 농사에 참여할 수 있는 **지속 가능한 환경**을 조성하고자 한다.

### 2) 향후 협력 전략

텃밭닥터 프로젝트의 지속 가능한 성장을 위해, 공공(B2G) 및 민간 부문(B2B & B2C)과의 유기적인 협력을 통해 도시농업 생태계를 확장하는 것을 핵심 전략으로 삼는다. 데이터 기반의 신뢰를 구축하고 사업 모델을 다각화하여, 대한민국 도시농업을 대표하는 통합 플랫폼으로 성장하고자 한다.

**가) 공공 부문 협력을 통한 사업 기반 강화 전략 (B2G)**
정부 및 지방자치단체의 도시농업 활성화 정책과 연계하여 공신력을 확보하고, 안정적인 B2G(Business-to-Government) 비즈니스 구조 기반을 마련한다.

**농촌진흥청** 및 **농업기술센터**와의 데이터 협력: 텃밭닥터의 실시간 사용자 데이터와 공공기관의 전문 연구 데이터를 교류하는 상생 모델을 구축한다. 앱의 AI 진단 정확도 향상과 함께 공공 부문에는 도시 환경 특화 농업 데이터 인사이트를 제공한다.

**지방자치단체(시·군·구)와의 플랫폼 연계**: 각 지자체 운영 공공 주말농장, 상자텃밭 보급 사업의 공식 애플리케이션으로 지정을 추진하고, 앱 내 공익적 배너 인프라를 통해 분양·교육·재해예방 공지를 맥락 맞춤으로 전달한다. 지자체는 시민 관리 효율성을, 텃밭닥터는 신뢰성 있는 사용자 기반을 확보하는 윈윈 구조를 구현한다.

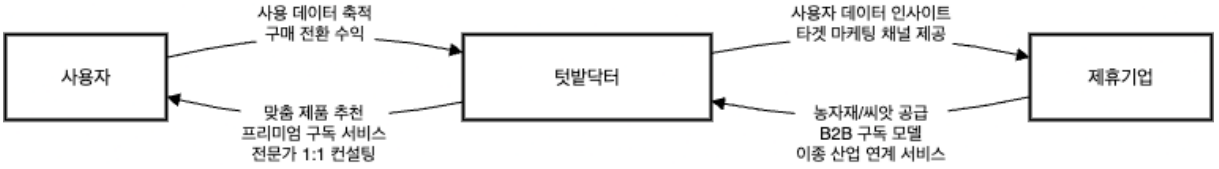
**나) 민간 부문 제휴를 통한 사업 모델 확장 전략 (B2B & B2C)**

다양한 산업군과의 연계를 통해 새로운 수익 모델을 창출하고, 플랫폼 가치를 극대화한다.

**농자재·씨앗 기업 제휴**: 앱 내 진단 결과에 따른 맞춤형 친환경 농약·비료·모종 추천 및 제휴사 쇼핑몰 연결을 통한 제휴 마케팅 모델을 개발한다.

**이종 산업(건설·교육) 연계**: 아파트 커뮤니티 텃밭, 학교 생태교육용 스마트가든 등에 텃밭닥터를 기본 관리 솔루션으로 탑재하는 B2B 구독 모델로 새로운 시장을 개척한다.

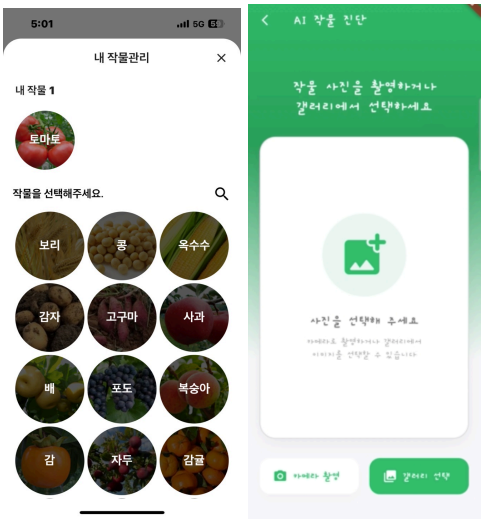
**개인 사용자 대상 서비스 확장(B2C)**:AI 영상 분석 정밀 진단, 전문가 1:1 화상 컨설팅 등 고도화 기능을 포함한 프리미엄 구독 모델을 도입해 지속 가능한 수익 기반을 구축한다.




텃밭닥터는 공공 부문 협력으로 신뢰도를 확보하고, 민간 제휴로 사업 모델을 다각화하여 도시농업 참여자·공공기관·관련 기업을 연결하는 **생태계 허브**로 자리매김한다.

### 3) 차별화 전략

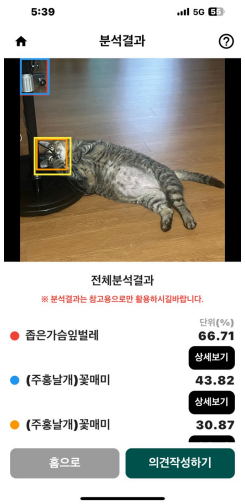
#### 가) 작물 미선택 프로세스




<스마트 병해충 진단 서비스> <텃밭닥터>

기존의 작물 병해충 진단 앱들은 사용자가 직접 작물의 종류를 선택한 후에야 병해충 진단 기능을 이용할 수 있는 구조를 가지고 있다. 이는 사용자가 작물에 대한 사전 지식을 갖추고 있어야 한다는 전제조건을 필요로 하며, 작물의 종류를 정확히 알지 못하는 초보 도시 농부에게는 진입 장벽으로 작용한다. 텃밭닥터는 이러한 불편함을 해소하고 사용자 편의성을 극대화하기 위해 **'작물 미선택' 진단 전략**을 채택했다. 초보자도 별도의 지식 없이 간편하게 병해 진단을 받을 수 있다는 점이 핵심 차별점이다.

#### 나) 작물 유효성 검증




<스마트 병해충 진단 서비스>

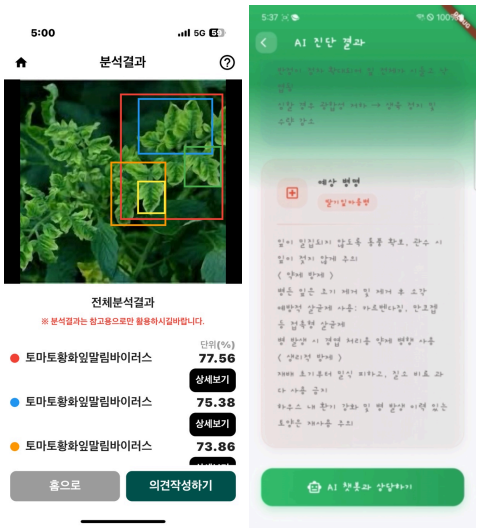
```
{
  "detail": "이미지에서 작물 관련 객체(식물, 잎, 과일 등)를 찾을 수 없습니다. 분석할 수 있는 다른 사진을 업로드해주세요."
}
```

<텃밭닥터>

대부분의 기존 작물 진단 앱은 사용자가 업로드하는 모든 이미지를 대상으로 진단 프로세스를 실행한다. 이 경우, 작물이 아닌 풍경, 사물, 사람 등의 사진이 업로드되더라도 시스템은 불필요한 분석을 시도하게 된다. 이는 무의미한 진단 결과를 생성하고 사용자에게 혼란을 줄 뿐만 아니라, 서버 및 AI 모델의 연산 자원(resource)을 심각하게 낭비하는 비효율을 초래한다.

텃밭닥터는 이러한 문제점을 해결하고 시스템의 안정성과 효율성을 확보하기 위해 **'작물 유효성 검증(Crop Validity Verification)' 전략**을 도입했다. 진단 프로세스 실행 전, 업로드된 이미지가 분석에 유효한지 사전에 판단하는 단계를 추가하여 **불필요한 리소스 낭비를 원천적으로 차단**한다. 이를 위해 저희는 YOLO 기반의 객체 탐지 기술을 활용한 작물 유효성 검증 모델을 개발하여 진단 시스템의 최전선에 배치했다.

### 다) 실시간 챗봇 상담



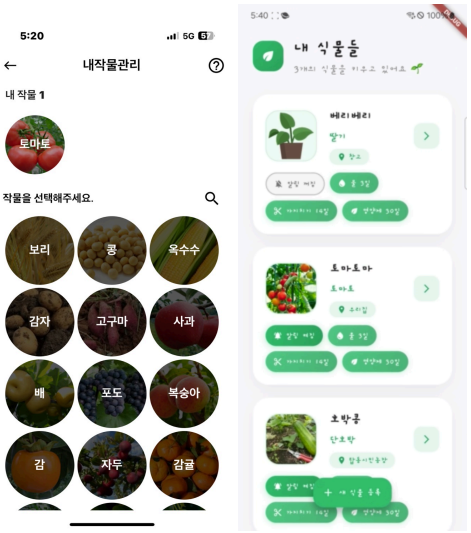

<스마트 병해충 진단 서비스> <텃밭닥터>

기존 작물 진단 앱들의 가장 큰 한계는 AI 진단이 완료된 후에 발생한다. 대부분의 앱은 진단 결과를 일방적으로 제시하고, '의견 작성하기'와 같은 수동적인 기능만을 제공할 뿐, 사용자의 후속 질문이나 궁금증을 해소할 수 있는 창구가 전무하다. 이는 진단 결과에 대한 추가 설명이나 구체적인 관리 방법에 대한 정보를 얻고 싶은 사용자, 특히 전문 지식이 부족한 초보 농부에게 큰 장벽으로 작용하며, 실질적인 문제 해결로 이어지지 못하는 '단절된 경험'을 제공한다.

텃밭닥터는 이러한 진단 후 소통의 부재를 해결하기 위해 **'실시간 챗봇 상담' 전략**을 도입하여,

진단에서 문제 해결까지 이어지는 **통합적인 사용자 경험**을 설계했다. AI 진단이 끝이 아니라, 사용자 맞춤형 케어의 시작점이 되도록 만드는 것이 본 전략의 핵심이다.

### 라) 효율적인 내 작물 관리




<스마트 병해충 진단 서비스>　　<텃밭닥터>

기존의 농업 관련 앱에서 제공하는 '내 작물 관리' 기능은 단순히 사용자가 재배 중인 작물 목록을 나열하는 수준에 그치는 경우가 대부분이다. 이는 사용자의 구체적인 재배 환경이나 개별 작물의 성장 과정을 체계적으로 기록하고 관리하는 데 명백한 한계를 가진다. 사용자는 여러 작물을 동시에 키우거나, 같은 작물을 다른 시기에 심었을 경우 이를 구분하고 관리하는 데 어려움을 겪게 된다.
텃밭닥터는 이러한 피상적인 관리 방식에서 벗어나, 사용자 개개인에게 최적화된 재배 경험을 제공하기 위해 **'효율적인 내 작물 관리' 전략을 수립**했다. 체계적인 설계를 통해 구현하여, 단순한 목록이 아닌 살아있는 **'디지털 농사 일지' 기능을 제공**한다.

## 3.1.4 기대효과

텃밭닥터는 사회적, 정책적, 경제적 측면에서 다음과 같은 종합적인 효과를 기대할 수 있다.

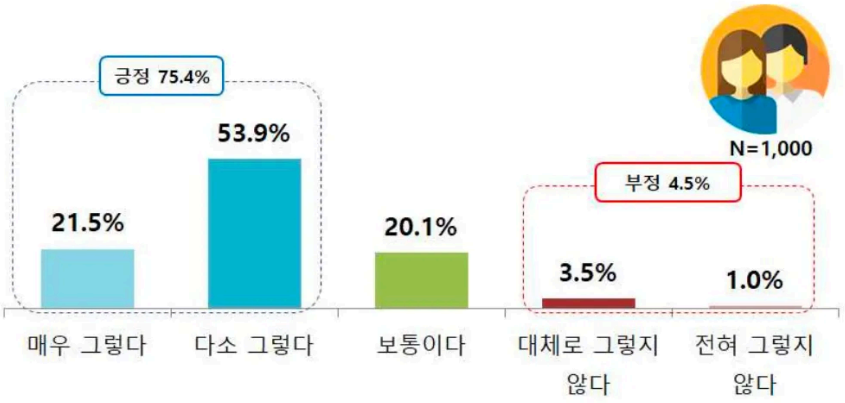
### 1) 사회적 측면

사회적 측면에서는 두 가지 주요 효과가 기대된다.  첫째, **신규 참여율이 50% 이상 증가**하는 효과를 가져올 것으로 예상된다. 둘째, 수요-공급 불균형 상황이 해소되어 **참여 희망자의 80% 이상**이 실제 참여로 전환되는 결과를 창출할 것이다.

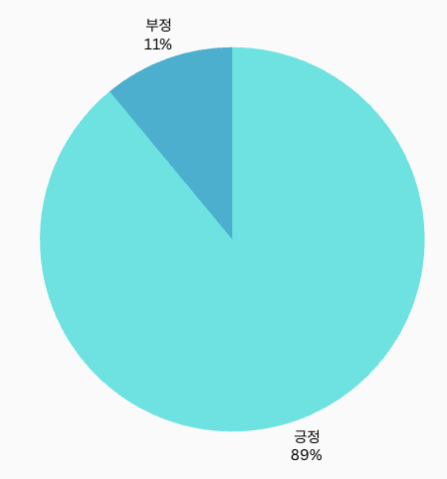
**도시농업에 대한 사회적 기반**

도시농업 확대 추진에 대해 **75.4%**가 **긍정적 반응**을 보였으며, 아파트 공유텃밭 **참여 의향은**

**89.2%**에 달한다. 서울시 공유텃밭의 **경쟁률 6:1**은 실증적 수요를 보여주는 핵심 지표로, 강력한 잠재 수요가 존재함을 입증한다.




<도시농업 확대 추진에 대한 응답>




<아파트 단지 내 공유텃밭 참여의향>

<서울시 공유텃밭 참여 경쟁률>

강력한 참여 수요를 바탕으로, 텃밭닥터는 신규 참여를 절반 이상 확대하고, 희망자의 80% 이상을 실제 참여로 전환시키는 실질적 성과를 창출할 것이다.

### 2) 정책적 측면

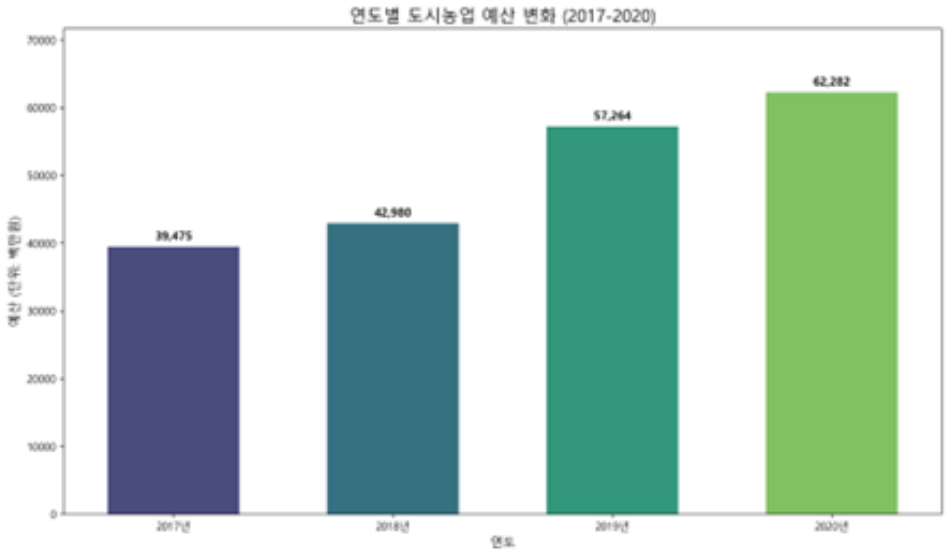
플랫폼 안정성과 공공 협력 확대에 상당한 효과가 기대된다. **일일 활성 사용자(DAU) 5,000명 달성**과 **지자체 공식 앱 지정 확대**를 통한 안정적 사용자 기반 구축이 가능할 것으로 예상된다.

**농업진흥구역 규제 완화에 따른 활용도 증대**

2025년 농업진흥구역 주말농장 **운영 규제 완화**로 도시농업 참여 기회가 확대되며, 텃밭닥터의 위치 기반 서비스를 통해 새롭게 개방된 농업 공간을 효율적으로 활용할 수 있게 되었다. 이러한 **공급 기반 확대**는 플랫폼 사용자 증가의 핵심 동력으로 작용할 것이다.

**정부 및 지자체 정책 지원**

농림축산식품부는 2024년 농업의 디지털전환, 세대전환, 농촌공간 전환 등 3대 전환 정책을 추진하고 있으며, 이 중 **디지털 전환과 농촌공간 전환 과제**가 텃밭닥터 프로젝트와 직접 연계된다.
도시농업 예산은 2017년 394억원에서 2020년 622억원으로 4년간 **약 58%** 증가하여 정부의 지속적인 투자 의지와 정책적 지원 기반이 충분히 조성되어 있음을 보여준다.




<출처 - 도시농업현황조사(농식품부과학기술정책과)>

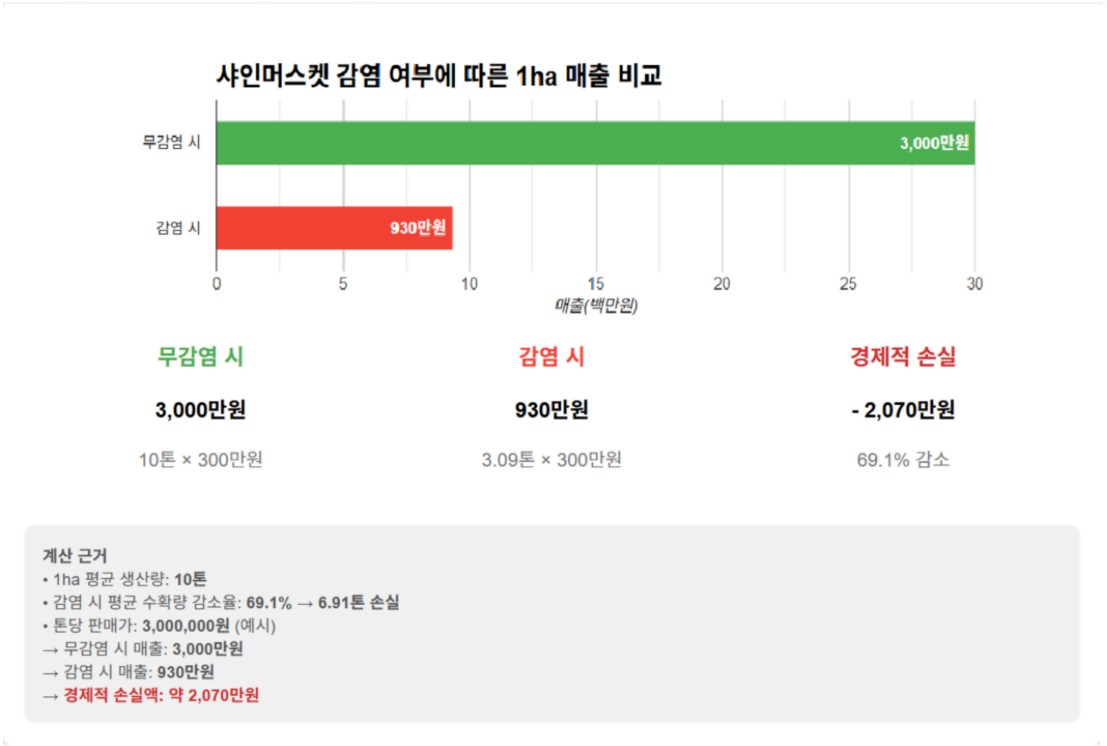
이러한 **규제 완화**와 **정책 지원 강화**라는 양방향의 정책적 뒷받침을 바탕으로, **DAU 5,000명 달성**과 **지자체 공식 앱 지정 확대**라는 효과는 충분히 실현 가능할 것으로 판단된다.

**3) 경제적 측면**

AI 기반 진단 시스템을 통한 직접적이고 실질적인 경제적 효과가 기대된다.
**작물 피해율 50% 이상 감소,** 참여 농가 평균 **생산량 30% 이상 향상**, 연간 **50~150억원의 경제적 파급효과 창출**이 가능할 것으로 예상된다.

**AI 기반 병해충 진단 시스템의 직접적 효과**

농촌진흥청 연구에 따르면, 샤인머스켓이 바이러스와 바이로이드에 감염될 경우 수확량이 **69% 감소**하며 1ha당 **2,070만원의 경제적 손실**이 발생한다.




<출처 - 농촌지방소식(농촌진흥청)>

텃밭닥터의 AI 진단 시스템은 작물 상태를 실시간으로 모니터링하고 병해를 조기에 발견함으로써 이러한 손실을 **최소화**할 수 있다.

**생산성 향상을 통한 경제적 파급효과**

실증 데이터 분석 결과, AI 시스템을 도입한 농가에서 생산량이 **평균 32.1% 증가**하였으며, 피해율이 **30~40% 감소**하는 효과가 확인되었다. 이를 통해 개별 농가 차원에서는 **생산성 향상**과 **비용 절감**에 직접적으로 기여할 것으로 예상된다.

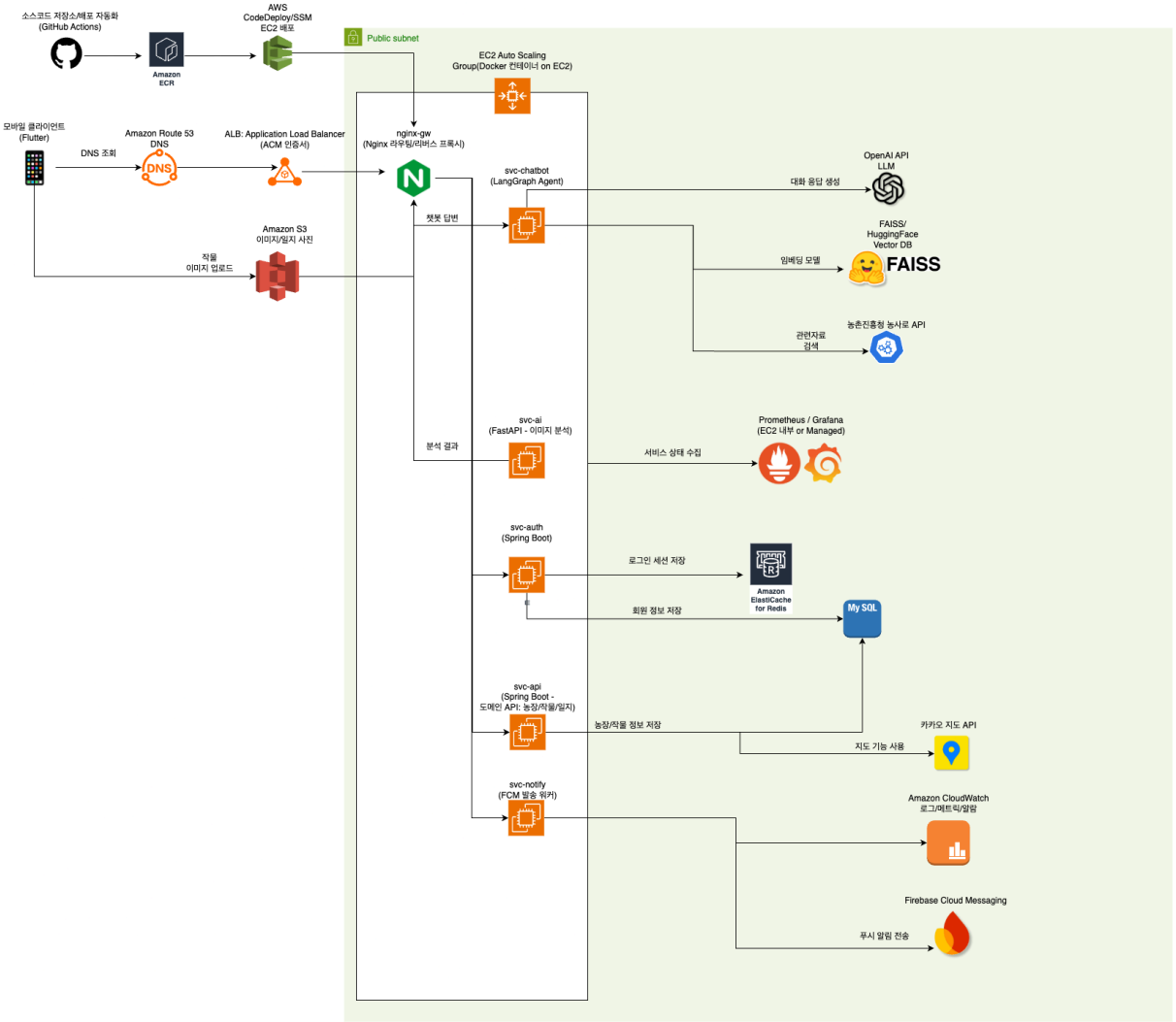
또한 병해로 인한 불확실성을 줄임으로써 **재배 안정성**과 **투자 효율성**이 향상되고, 주말농장이나 소규모 농장에서도 **참여자 만족도 증가**로 이탈율 감소와 운영 안정성 확보 효과를 가져올 것이다.

도시농업 참여자 규모(174만명)와 텃밭닥터 예상 사용률(10~30%)을 고려할 때, 연간 **50~150억원**의 **경제적 파급효과 창출**이 가능할 것으로 분석된다.

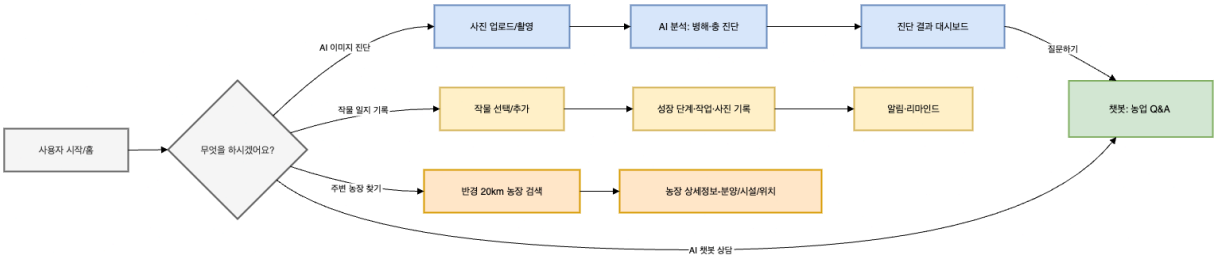
## 3.2 기술 구현 및 설계

### 3.2.1 전체 시스템 아키텍처

#### 가) 시스템 구성도



### 나) 사용자 플로우차트




사용자가 앱 실행 후 선택할 수 있는 네 가지 주요 경로를 보여준다.

△ **AI 이미지 진단** : 사진 업로드로 작물 병해충 분석하고 결과에 따라 상담으로 연결된다.

△ **AI 챗봇 상담** : 24시간 Q&A, 농업 기술 상담기능 제공.

△ **작물 일지 기록** : 성장 단계와 작업 이력을 기록하고 알림으로 체계적 관리 지원.

△ **주변 농장 찾기** : 반경 20km 내 농장을 검색해 분양·시설 현황 등 상세 정보를 확인할 수 있다.

### 3.2.2 기술 스택 구성

| 구분 | 내용 |
|---|---|
| **AI/ML 기술** | PyTorch: 연구→서빙 전환 용이<br>convNext-tiny: 이미지 병해 분류 성능/효율 균형<br>LangGraph: 도구 호출·대화 흐름 제어(RAG/에이전트)<br>임베딩 모델 – sentence-transformers/all-MiniLM-L6-v2 (Hugging Face) |
| **LLM & 임베딩** | OpenAI API(LLM) : LLM: 자연어 이해/생성<br>Hugging Face 임베딩 모델 : 가격, 문서 임베딩 품질<br>FAISS(Vector DB) : 대용량 시맨틱 검색 |
| **Language** | Python : ML/서빙<br>Java 17 : 안정적 생태계<br>Dart : 크로스플랫폼 모바일 |
| **Framework** | FastAPI : 경량·고성능 AI 서빙/REST<br>Spring Boot : 인증·도메인 API 표준화<br>Flutter : 단일 코드베이스로 iOS/Android 동시 개발 |

| | |
|---|---|
| **Library** | Spring Data JPA, Spring Web (MVC)<br>Spring Security: 인증/인가, SpringDoc(Swagger),<br>Spring WebFlux (WebClient) : 비동기 통신<br>AWS S3, OAuth 2.0<br>JWT(JSON Web Token), Thymeleaf |
| **API** | OpenAI API<br>카카오 지도 : 위치/지도 UI<br>FCM(Firebase Cloud Messaging) : 푸시 알림 인프라<br>농사로(Open API) : 전문 DB 구축 |
| **DB** | MySQL : 사용자/작물/농장 **메인 운영 DB**<br>Redis : 인메모리 기반 DB, 캐싱 및 세션 관리<br>FAISS(Vector DB) : 임베딩 검색<br>SQLite :FastAPI 모듈의 진단 기록·챗 히스토리(경량·무설치) |
| **IDE** | IntelliJ, VS Code, Android Studio |
| **Tools** | 카카오톡, Slack, Notion, Github, Figma, Postman |
| **인프라 & 배포** | AWS, Docker, Gradle |
| **DataSet** | 경기데이터드림 텃밭 분양 정보 데이터셋,<br>농림축산식품 공공데이터 포털 텃밭 분양 상세정보 데이터셋,<br>AI Hub 시설작물 질병 진단 이미지 데이터셋 |

### 3.2.3 계층 구조 및 모듈화 설계

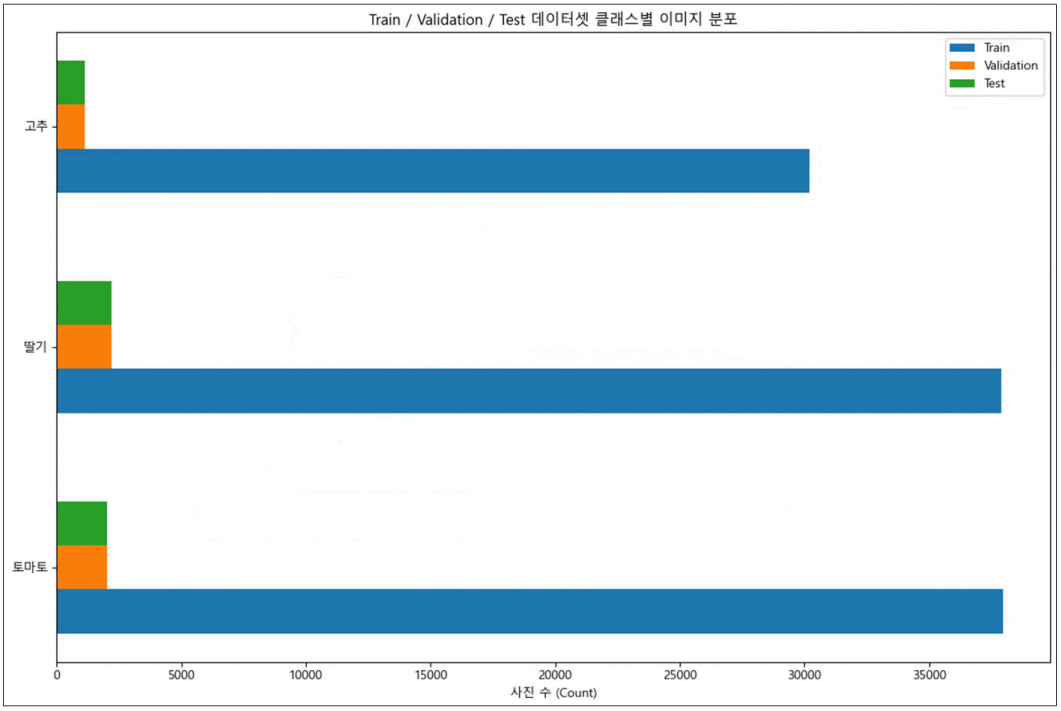
| 분류 | 구성요소 | 주요 역할 |
|---|---|---|
| **Client Layer** | Mobile App (Flutter) | 도시농업 서비스 제공 (진단, 지도, 채팅 등), 모바일 최적화 앱, 오프라인 지원 |
| | User Dashboard | 사용자 맞춤 농작물 일지 관리, 정보 확인 |
| | Admin Page | 관리자 페이지(User, Plant, Farm, Notice 관리 - 등록, 조회, 수정, 삭제, 알림 발송) |
| **API Gateway** | Nginx / Load Balancer | 클라이언트 요청 분산 및 백엔드 라우팅 |
| **Backend Services** | Auth Service | 사용자 인증/인가, 로그인/회원가입, JWT 관리, 사용자 정보 관리 |
| | Image Analysis Service | 작물 사진 분석, 병해충 AI 진단 (FastAPI) |
| | Chat Service | 실시간 상담, LangGraph 기반 질의응답 |
| | Farm Service | 관리자: 농장 관리 기능, 사용자: 조회, 검색, 위치 기반 주말농장 정보 제공 |
| | Plant Service | 관리자: 작물 종류 관리 기능, 사용자: 조회, 검색 |
| | UserPlant Service | 사용자가 재배하는 작물 정보 관리, 조회, 검색 |
| | Diary Service | 사용자 등록 식물에 대한 일지 관리 및 기록 |
| | Notification & Notice Service | 푸시 알림, 정책 소식, 일정 안내 (FCM) 및 공지사항 관리 |
| | Image File Service | 이미지를 사용하는 모든 도메인의 이미지 파일 |

| | | |
|---|---|---|
| | | 관리 |
| **Data Layer** | MySQL | 사용자 정보, 작물 일지, 농장 정보, 작물 정보 등 관계형 데이터 저장 |
| | FAISS(Vector DB) | 농업 지식 임베딩 저장, 챗봇 응답 근거 |
| | Redis | 세션 관리 및 캐싱 |
| | File Storage (AWS S3) | 사용자, 작물, 농장, 병해충 분석 이미지 및 일지 사진 저장 |
| | External APIs | 농진청, FCM, 지도 API 등 외부 연계 |
| | Monitoring (Prometheus / Grafana) | 서비스 상태 모니터링 및 로그 분석 |
| **AI/ML Mode Serving** | ConvNeXt | CNN 모델 기반 이미지 병해충 분석 |
| | GPT API | LLM 모델 (추가 응답 보조용) |
| | LangGraph | 대화 흐름 제어 및 맞춤형 상담 제공 |

### 3.2.4 AI 구성

#### 1) 1차 작물 분류 모델

#### 가) 사전 정보 수집 및 전처리




- **고품질 데이터 확보 및 구성:** 모델 학습의 기반이 되는 데이터는 신뢰성 높은 공공 데이터인 AI-Hub의 **'농작물 병해 진단 이미지'** 데이터셋을 활용했다. 원본 이미지 각 약 11,000장과 데이터 증강(Augmentation) 기법이 적용된 이미지 약 21,000장을 포함, 총 32,000장 이상의 대규모 데이터셋을 구축하여 모델이 다양한 환경의 작물을 학습할 수 있도록 했다.
- **엄격한 데이터셋 분할 및 분포 시각화:** 모델의 성능을 객관적으로 평가하기 위해 전체 데이터를 학습(Train), 검증(Validation), 테스트(Test) 용도로 8:1:1 비율로 무작위 분할했다. 이 과정에서 각 데이터셋의 클래스별 분포가 균일하게 유지되었는지 확인하기 위해, Pandas 라이브러리로 데이터 통계를 집계하고 Matplotlib과 Seaborn을 이용해 분포도를 시각적으로 확인함으로써 데이터 편향의 위험을 최소화했다.
- **표준화된 이미지 변환:** 모든 이미지는 전이 학습에 사용할 EfficientNetV2-M 모델의 입력 크기에 맞춰 224x224 픽셀로 일괄 변환했다. 또한, 이미지 픽셀 값을 PyTorch가 처리할 수 있는 Tensor 자료구조로 변환하고, ImageNet 데이터셋의 평균과 표준편차로 **정규화(Normalization)**하는 과정을 거쳤다. 이는 사전 훈련된 모델의 성능을 최대한 활용하기 위한 필수적인 전처리 단계이다.
- **클래스 불균형 해소를 위한 가중치 계산 및 적용:** 데이터 분포 분석 결과, 클래스별 이미지 수에 불균형이 존재함을 확인했다. 모델이 데이터 수가 많은 클래스에만 편향되어 학습하는 것을

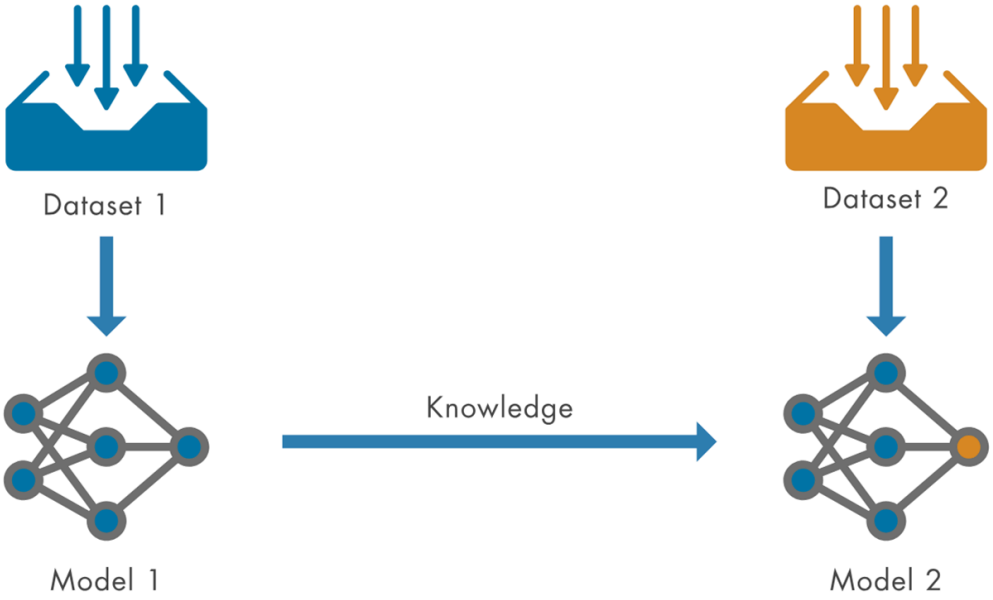
방지하기 위해, 학습 데이터셋의 클래스별 이미지 수의 역수를 취하는 방식으로 **클래스 가중치(Class Weights)**를 계산했다. 이 가중치를 손실 함수인 CrossEntropyLoss에 적용함으로써, 데이터 수가 적은 소수 클래스의 학습 중요도를 인위적으로 높여 **모든 클래스를 균형 있게 학습**하도록 유도했고, 이를 통해 모델의 전반적인 분류 성능을 향상시켰다.

```
# --- 클래스 가중치 계산 및 적용 ---
print("클래스 가중치 계산을 시작합니다...")

# 1. 훈련 데이터셋의 클래스별 데이터 수 계산
# train_dataset.targets는 각 이미지의 레이블(0, 1, 2...)을 담고 있는 리스트입니다.
class_counts = Counter(train_dataset.targets)
class_counts = [class_counts[i] for i in range(len(train_dataset.classes))] # 클래스 인덱스 순으로 정렬

print("\n훈련 데이터셋의 클래스별 데이터 수:")
for i, count in enumerate(class_counts):
    print(f"- {train_dataset.classes[i]}: {count}장")


num_total_samples = len(train_dataset)
weights = [num_total_samples / count for count in class_counts]

weights_tensor = torch.FloatTensor(weights)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
weights_tensor = weights_tensor.to(device)
print(f"\n현재 사용 중인 디바이스: {device}")

criterion = torch.nn.CrossEntropyLoss(weight=weights_tensor)

print("\n계산된 클래스 가중치:")
print(weights_tensor)
print("\nCrossEntropyLoss에 가중치가 성공적으로 적용되었습니다.")
print("이제부터 모델 학습 시 이 'criterion' 변수를 손실 함수로 사용하세요.")
```

*# 클래스 가중치 계산 및 적용*

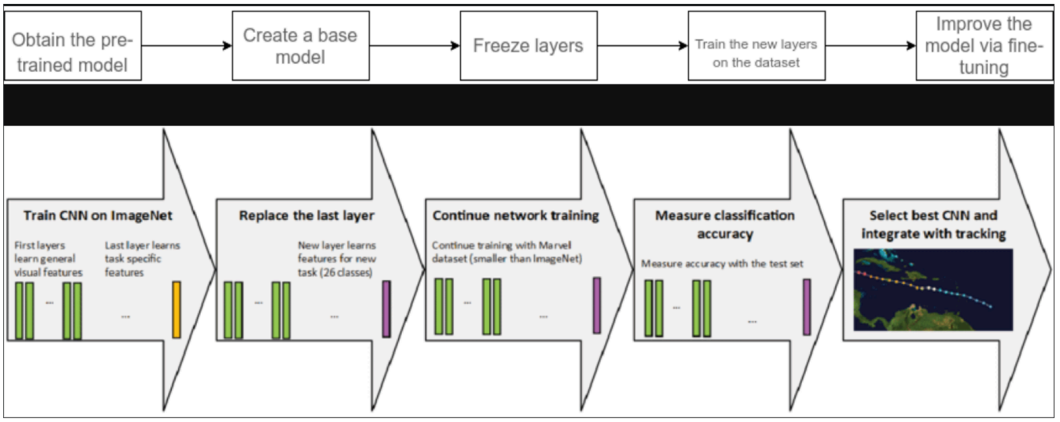
### 나) 모델 선정 및 개요

- **전이 학습의 개념과 효용성: 전이 학습(Transfer Learning)**은 ImageNet과 같은 대규모 데이터셋으로 사전에 훈련된 모델이 학습한 방대한 시각적 지식(가중치)을 새로운 과제(작물 분류)에 재활용하는 기법이다. 이는 수백만 장의 이미지를 통해 학습된 범용적인 특징 추출(Feature Extraction) 능력을 그대로 활용하므로, 제한된 데이터로 처음부터 모델을 훈련시키는 것보다 월등히 높은 성능을 더 빠른 시간 안에 달성할 수 있는 핵심적인 전략이다.




# Transfer Learning - MATLAB & Simulink

- **EfficientNetV2-M 모델 아키텍처 적용:** 본 프로젝트에서는 현재 가장 뛰어난 성능과 효율성을 보이는 모델 중 하나인 **EfficientNetV2-M**을 기반 모델로 채택했다. 사전 훈련된 모델을 불러온 후, 모델의 대부분을 차지하는 **특징 추출 부분(Feature Extractor)의 모든 가중치를 동결(requires_grad=False)**시켰다. 이는 기존 모델이 학습한 선, 질감, 형태 등 이미지의 저수준 및 고수준 특징을 보존하기 위함이다. 그 후, 기존 모델의 분류기(Classifier)를 '텃밭닥터'의 목표 클래스(고추, 딸기, 토마토 등) 수에 맞는 **새로운 nn.Linear 레이어로 교체**했다. 이 과정을 통해, 강력한 범용 특징 추출기는 그대로 유지한 채, 최종 판단을 내리는 분류기 부분만 우리의 특정 과제에 맞게 새로 훈련시키는 효율적인 **미세 조정(Fine-tuning)**을 구현했다.




# Transfer Learning Guide: A Practical Tutorial With Examples for Images and Text in Keras

### 다) 모델 분석

- **가중 교차 엔트로피 손실(Weighted CrossEntropyLoss) 사용:** 데이터 전처리 과정에서 확인된 클래스 간 불균형 문제를 해결하기 위해, 데이터 수가 적은 클래스에 더 높은 가중치를 부여하는 손실 함수를 사용했다. 이를 통해 모델이 모든 작물 클래스를 편향 없이 균등하게 학습하도록 유도했다.
- **안정적인 학습을 위한 최적화 설정:** 옵티마이저로 **AdamW**를 사용하고, 학습률 스케줄러로 **ReduceLROnPlateau**를 적용했다. 이는 검증 데이터에 대한 손실(validation loss)이 일정 기간 개선되지 않으면 학습률을 자동으로 낮추어, 모델이 최적점에 더 안정적으로 수렴하도록 돕는 전략이다.
- **최고 성능 모델 저장 및 학습 재개:** 학습 중 검증 손실이 가장 낮아지는 시점의 모델 가중치를 '최고의 모델'로 간주하고 **별도 저장(Checkpoint)**했다. 이를 통해 과적합을 방지하고 가장 일반화 성능이 뛰어난 모델을 최종적으로 선택할 수 있도록 했다. 또한, 학습이 중단되더라도 저장된 시점부터 이어서 훈련을 재개할 수 있는 기능을 구현하여 학습 효율성을 높였다.

### 라) 모델 개선

- **차등 학습률(Differential Learning Rates) 적용:** 초기 학습으로 확보된 최고 성능 모델을 기반으로, **모델의 마지막 2개 블록과 분류기(Classifier)만 학습을 재개(Unfreeze)하는 고급 파인튜닝**을 진행했다. 이때, 기존 지식의 손실을 최소화하기 위해 이미지의 복잡한 특징을 학습하는 모델의 몸통(Body) 부분은 매우 낮은 학습률(1e-6)을, 새로운 분류 과제에 맞춰진 머리(Head) 부분은 상대적으로 높은 학습률(1e-5)을 적용하는 차등 학습률 전략을 사용하여 모델을 더욱 정교하게 미세 조정했다.

```
# --- 🧠 3. 모델 준비: 로드 및 고정 해제(Unfreeze) ---
print("🚀 3. 모델 로드 및 레이어 고정 해제")
model = models.efficientnet_v2_m(weights=None)
num_ftrs = model.classifier[1].in_features
model.classifier[1] = nn.Linear(num_ftrs, NUM_CLASSES)
model.load_state_dict(torch.load(INPUT_MODEL_PATH, map_location=DEVICE))
print(f"   - ✅ '{INPUT_MODEL_PATH}' 모델을 성공적으로 로드했습니다.")

for param in model.parameters(): param.requires_grad = False
for param in model.features[-1].parameters(): param.requires_grad = True
for param in model.features[-2].parameters(): param.requires_grad = True
for param in model.classifier.parameters(): param.requires_grad = True
model.to(DEVICE)
print("   - 마지막 2개 블록 및 분류기 레이어의 학습을 활성화했습니다.")
```

- **데이터 증강 및 과적합 방지 기법 고도화:** 훈련 데이터에 **ColorJitter**와 같은 추가적인 **데이터 증강(Augmentation)** 기법을 적용하여 모델이 더 다양한 조명과 색상 변화에 강건하게 반응하도록 했다. 또한, 모델이 정답을 과도하게 확신하여 발생하는 과적합을 방지하기 위해 **레이블 스무딩(Label Smoothing)**을 손실 함수에 적용하여 일반화 성능을 높였다. 학습률 스케줄러로는 **코사인 어닐링(Cosine Annealing)** 방식을 사용하여, 학습이 진행됨에 따라 학습률이 부드럽게 감소하며 최적의 해에 안정적으로 도달하도록 개선했다.

```
# --- 🏞️ 2. 데이터 준비 ---
print("🚀 2. 데이터 로더 생성")
data_transforms = {
    # [수정] 훈련 데이터에 데이터 증강(Augmentation) 적용
    'train': transforms.Compose([
        transforms.RandomResizedCrop(224, scale=(0.8, 1.0)),
        transforms.RandomHorizontalFlip(),
        transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ]),
    'valid': transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ]),
}
```

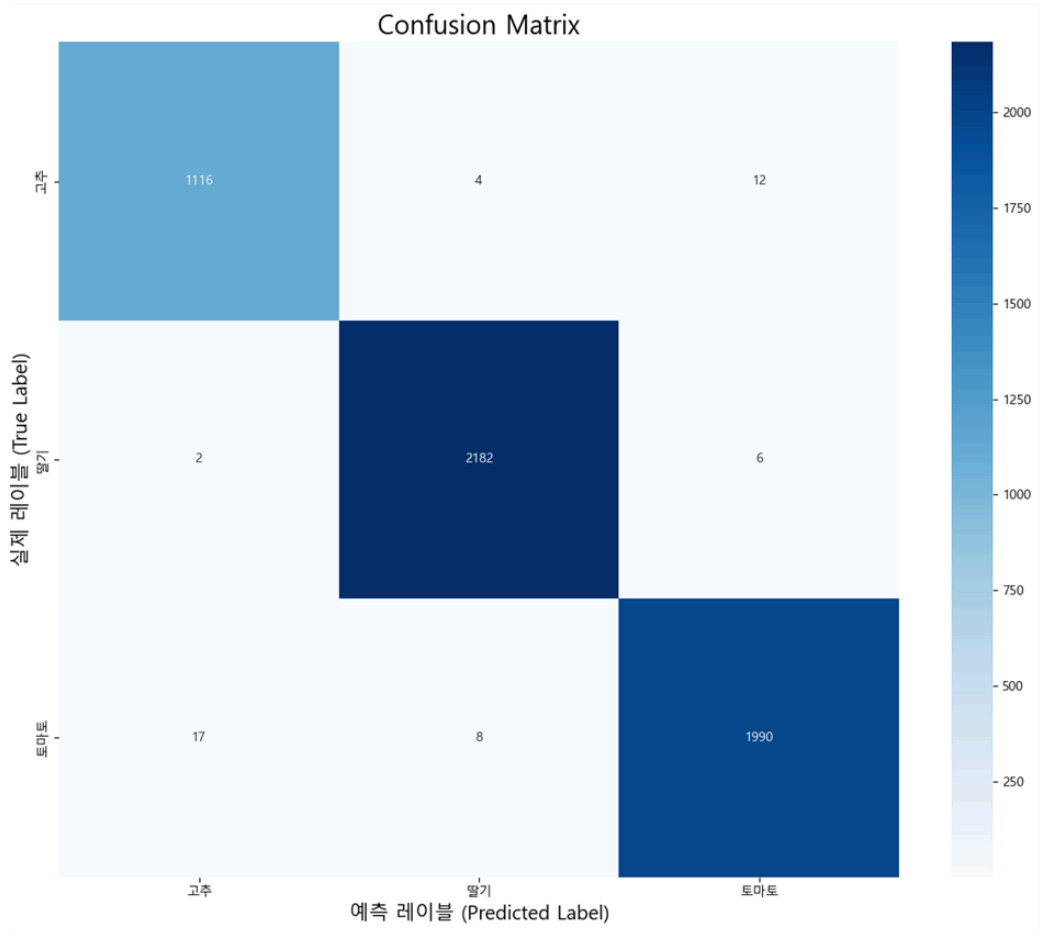
### 마) 모델 평가

- 최종적으로 완성된 작물 분류 모델의 객관적인 성능을 측정하기 위해, 학습 및 검증 과정에서 전혀 사용되지 않은 5,337장의 별도 테스트 데이터셋으로 평가를 진행했다. 평가 결과, 모델은 **전체 정확도(Accuracy) 99.08%** 라는 매우 높은 성능을 기록했다.
- 각 클래스별 성능을 상세히 분석한 결과, **정밀도(Precision), 재현율(Recall), F1-Score 모든 지표에서 98% 이상의 높은 수치를 보였다.** 특히 데이터 수가 가장 많은 '딸기' 클래스는 물론, 상대적으로 수가 적은 '고추'와 '토마토' 클래스에서도 편향 없는 우수한 성능을 나타냈다. 이는 모델이 특정 클래스에 과적합되지 않고 모든 작물을 안정적으로 분류하는 높은 일반화 성능을 갖추었음을 의미하며, 실제 서비스 환경에서의 높은 신뢰도를 기대할 수 있는 결과이다.

- 비록 시간 및 자원 제약으로 인해 대표적인 3개의 핵심 작물 클래스에 대한 모델을 우선 개발하였으나, **이의 성공적인 성능 검증을 통해 향후 전체 12개 작물 클래스로 확장 가능한 기술적 타당성과 잠재력을 충분히 입증하였다.**

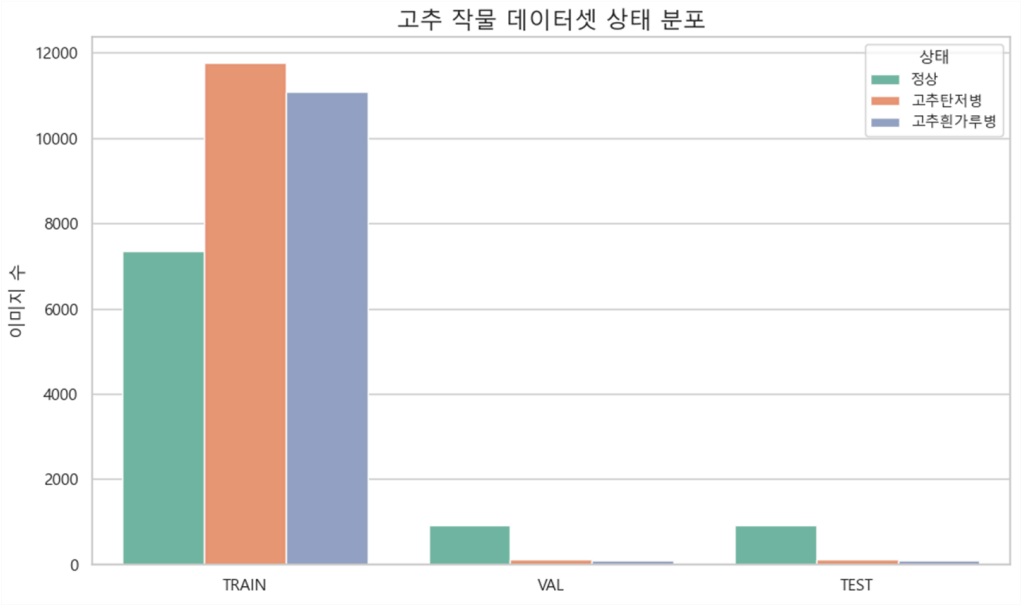
**Classification Report**

| Class | Precision | Recall | F1-Score | Support |
|---|---|---|---|---|
| 고추 | 0.983 | 0.986 | 0.985 | 1,132 |
| 딸기 | 0.995 | 0.996 | 0.995 | 2,190 |
| 토마토 | 0.991 | 0.988 | 0.989 | 2,015 |
| accuracy | | | 0.991 | 5,337 |
| macro avg | 0.99 | 0.99 | 0.99 | 5,337 |
| weighted avg | 0.991 | 0.991 | 0.991 | 5,337 |



### 2) 2차 작물 병해 분류 모델 (대표 예시로 '고추'모델 사용)

#### 가) 사전 정보 수집




- **고품질 공공 데이터셋 확보 및 클래스 정의:** 모델의 신뢰성과 재현성을 확보하기 위해 공공 데이터 포털인 AI-Hub의 '**노지 작물 질병 진단 이미지**'를 원천 데이터로 활용했다. 이 중 '고추' 관련 데이터셋을 대상으로, 제공된 파일명의 명명 규칙을 분석하여 '정상', '고추탄저병', '고추흰가루병'의 3가지 핵심 클래스를 프로그램적으로 정의하고 자동 분류했다. 이는 수작업 라벨링으로 인한 오류를 최소화하고 대용량 데이터를 효율적으로 처리하기 위함이다.
- **데이터 유출 방지를 위한 엄격한 데이터셋 분할:** 모델의 일반화 성능을 정확하게 평가하기 위한 핵심 전처리 과정으로, 데이터 증강(Augmentation)을 적용하기 전에 원본 이미지만을 대상으로 데이터셋 분할을 우선 수행했다. 원본 이미지를 랜덤 셔플링한 후, 8:1:1의 비율에 따라 학습(Train), 검증(Validation), 테스트(Test) 데이터셋으로 분리했다. 이처럼 원본 데이터를 먼저 분리하고 증강 데이터를 나중에 추가하는 전략은, 검증 및 테스트 데이터의 변형 이미지가 학습 데이터에 포함되는 **데이터 유출(Data Leakage) 현상을 원천적으로 방지**하여 평가의 신뢰도를 확보하는 데 결정적인 역할을 한다.
- **학습 데이터 강화를 위한 증강 데이터 활용:** 데이터셋 분할이 완료된 후, 모델의 강건성(Robustness)과 성능을 높이기 위해 미리 생성된 모든 '증강' 이미지를 학습(Train) 데이터셋에만 추가했다. 이를 통해 모델은 원본 이미지에 없는 다양한 구도, 밝기, 색상 변화 등을 학습하여 실제 환경에서 마주할 수 있는 여러 변수에 효과적으로 대응할 수 있게 된다. 반면, 검증 및 테스트 데이터셋은 순수한 원본 이미지만으로 유지하여, 학습되지 않은 실제 데이터에 대한 모델의 성능을 객관적으로 측정할 수 있도록 구성했다.

### 나) 데이터 전처리 및 증강

- **데이터 불균형 해소를 위한 클래스 가중치 적용:** '정상' 클래스의 이미지 수가 다른 질병 클래스에 비해 상대적으로 적은 데이터 불균형 문제를 해결하기 위해 **클래스 가중치(Class Weights) 기법을 적용**했다. 각 클래스에 속한 이미지 수의 역수를 취해 가중치를 계산하고, 이를 손실 함수(CrossEntropyLoss)에 적용했다. 이 방식은 데이터 수가 적은 소수 클래스의 중요도를 높여, 모델이 다수 클래스에 편향되지 않고 모든 병해를 균등하게 학습하도록 유도하여 전반적인 진단 성능을 향상시킨다.
- **이미지 품질 개선:** 먼저, **CLAHE(Contrast Limited Adaptive Histogram Equalization) 기법을 적용**하여 이미지의 국소적 대비를 선명하게 만들어 병반이 잘 드러나도록 했다. 이를 위해 이미지를 LAB 색 공간으로 변환 후, 밝기 채널(L)에만 적용하여 색상 왜곡을 최소화했다. 이후 **가우시안 블러(Gaussian Blur)**를 미세하게 적용하여 이미지의 노이즈를 제거하고 모델이 병징의 핵심 특징에 더 집중하도록 유도했다.

```python
class ApplyCLAHE:
    def __init__(self, clip_limit=2.0, tile_grid_size=(8, 8)):
        self.clip_limit = clip_limit
        self.tile_grid_size = tile_grid_size

    def __call__(self, pil_img):
        clahe = cv2.createCLAHE(clipLimit=self.clip_limit, tileGridSize=self.tile_grid_size)
        cv_img = cv2.cvtColor(np.array(pil_img), cv2.COLOR_RGB2BGR)
        lab_img = cv2.cvtColor(cv_img, cv2.COLOR_BGR2LAB)
        l, a, b = cv2.split(lab_img)
        l_clahe = clahe.apply(l)
        lab_img_clahe = cv2.merge((l_clahe, a, b))
        bgr_img_clahe = cv2.cvtColor(lab_img_clahe, cv2.COLOR_LAB2BGR)
        return Image.fromarray(cv2.cvtColor(bgr_img_clahe, cv2.COLOR_BGR2RGB))

# ApplyGaussianBlur는 보통 문제가 없지만, 만약 비슷한 오류가 발생한다면
# 이 클래스도 위와 동일한 방식으로 수정할 수 있습니다.
class ApplyGaussianBlur:
    """
    PIL 이미지를 입력받아 가우시안 블러를 적용하여 노이즈를 제거합니다.
    """
    def __init__(self, kernel_size=(5, 5), sigma_x=0):
        self.kernel_size = kernel_size
        self.sigma_x = sigma_x

    def __call__(self, pil_img):
        cv_img = cv2.cvtColor(np.array(pil_img), cv2.COLOR_RGB2BGR)
        blurred_img = cv2.GaussianBlur(cv_img, self.kernel_size, self.sigma_x)
        return Image.fromarray(cv2.cvtColor(blurred_img, cv2.COLOR_BGR2RGB))
```

- **일반 데이터 증강:** 모델의 일반화 성능과 강건성을 높이기 위해 다양한 데이터 증강 기법을 적용했다. **RandomHorizontalFlip(좌우 반전)**, **RandomRotation(회전)**, **ColorJitter(색상 변환)** 등 기본적인 기법과 더불어, 최신 증강 기법인 RandAugment를 도입하여 자동으로 최적의 증강 조합을 탐색하고 적용함으로써 데이터의 다양성을 극대화했다.

```
# 2. Transform 정의
train_transform = transforms.Compose([
    ApplyCLAHE(clip_limit=2.0, tile_grid_size=(8,8)),
    ApplyGaussianBlur(kernel_size=(3,3)),
    transforms.Resize((256, 256)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(20),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
    transforms.RandAugment(num_ops=2, magnitude=9),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

val_transform = transforms.Compose([
    ApplyCLAHE(clip_limit=2.0, tile_grid_size=(8,8)),
    ApplyGaussianBlur(kernel_size=(3,3)),
    transforms.Resize((256, 256)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])
```

- **고급 정규화 기법:** 과적합을 효과적으로 억제하기 위해 **CutMix와 MixUp**을 확률적으로 적용했다. 이 기법들은 두 개 이상의 이미지를 섞어 새로운 학습 데이터를 생성함으로써, 모델이 이미지의 특정 부분에만 과도하게 의존하는 것을 방지하고 더 강건한 특징을 학습하도록 돕는다. 또한, 손실 함수에는 **레이블 스무딩(Label Smoothing)**을 적용하여 모델이 정답 레이블을 100% 확신하지 않도록 규제함으로써 과적합을 방지하고 성능을 추가적으로 개선했다.

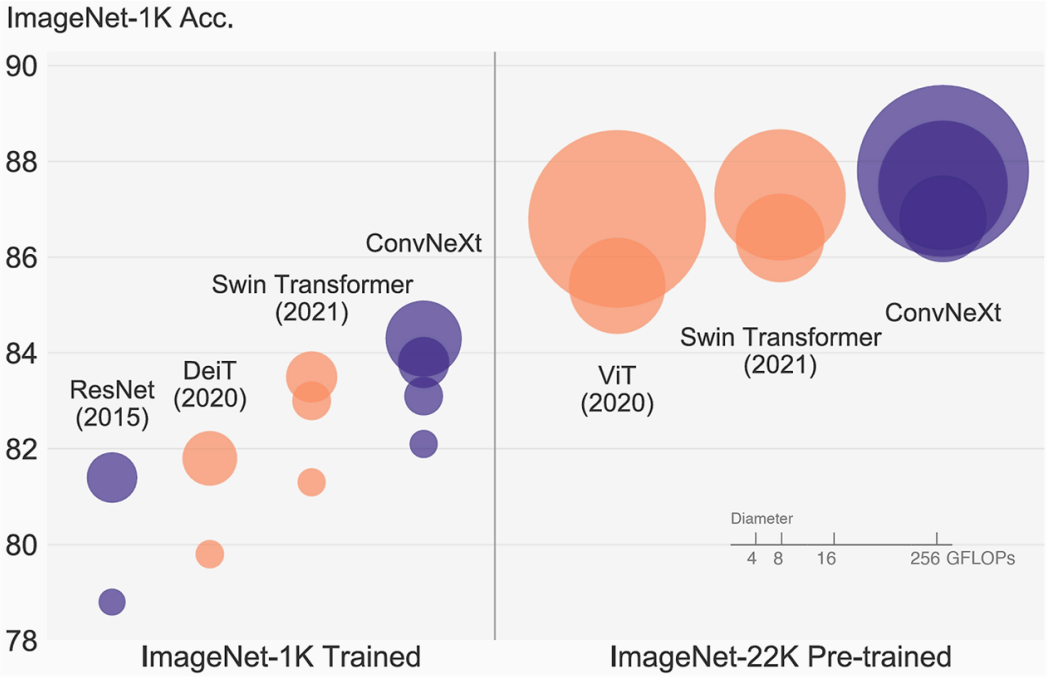
```
# --- 3. CutMix와 Mixup 변환 정의 ---
cutmix = T.CutMix(num_classes=num_classes, alpha=CONFIG["alpha"])
mixup = T.MixUp(num_classes=num_classes, alpha=CONFIG["alpha"])
# 둘 중 하나를 랜덤하게 선택하여 적용
cutmix_or_mixup = T.RandomChoice([cutmix, mixup])
```

**다) 모델 선정 및 개요**

- **전이 학습 기반의 ConvNeXt-Tiny 모델 아키텍처 적용:** 1차 분류 모델과 마찬가지로 **전이**

**학습(Transfer Learning) 기법**을 적용하여 개발 효율성과 성능을 동시에 확보했다. 기반 모델로는 기존 CNN의 장점과 Vision Transformer의 구조적 특징을 결합하여 높은 성능을 내는 **최신 아키텍처인 ConvNeXt-Tiny**를 채택했다. 이 모델은 ImageNet과 같은 대규모 데이터셋으로 사전 훈련된 가중치를 가져와, 소규모의 특정 데이터셋(고추 질병)에 맞게 재학습시키는 전이 학습에 매우 효과적이다.

- 이 모델은 전통적인 CNN의 효율성과 단순함을 유지하면서도, **Vision Transformer의 핵심 아이디어들을 점진적으로 도입**하여 성능을 극대화한 것이 특징이다. 구체적으로는, **전체 네트워크의 연산 분포를 재조정하고(Changing stage compute ratio)**, 연산 효율을 높이기 위해 **Depthwise Convolution**을 도입했으며, 채널 수를 확장했다가 다시 줄이는 **Inverted Bottleneck** 구조를 적용했다. 특히, 기존 CNN의 작은 커널(3x3) 대신 더 넓은 영역의 특징을 학습할 수 있도록 커널 크기를 7x7로 대폭 키우고, 배치 정규화(Batch Normalization) 대신 **계층 정규화(Layer Normalization)**를 사용하는 등 **Transformer의 설계 철학을 적극적으로 차용하여 기존 CNN 모델들의 성능 한계를 뛰어넘도록 설계**되었다.




**라) 모델 분석**

- **손실 함수 및 최적화 기법:** 모델 학습에는 **레이블 스무딩(Label Smoothing)**이 적용된 교차 엔트로피 손실(CrossEntropyLoss) 함수를 사용했다. 레이블 스무딩은 모델이 과도하게 정답을 확신하는 것을 방지하여 일반화 성능을 높이는 정규화 기법이다. 최적화

알고리즘으로는 ConvNeXt와 같은 최신 아키텍처에서 안정적이고 우수한 성능을 보이는 **AdamW 옵티마이저**를 채택하고, **학습률은 3e-4**로 설정하여 효율적인 학습을 유도했다.

```
# --- 4. ConvNeXt 모델 로드, 손실 함수, 옵티마이저 ---
model = models.get_model(CONFIG["model_name"], weights='IMAGENET1K_V1')
model.classifier[-1] = nn.Linear(model.classifier[-1].in_features, num_classes)
model.to(CONFIG["device"])

criterion = nn.CrossEntropyLoss(label_smoothing=0.1) # Soft Label에 더 강인하도록 Label Smoothing 적용
optimizer = optim.AdamW(model.parameters(), lr=CONFIG["learning_rate"])
```

- **최적 모델 선정 방식:** 총 20회의 Epoch 동안 학습을 진행하며, 각 Epoch가 끝날 때마다 검증 데이터셋(Validation set)으로 모델의 정확도를 평가했다. 학습 과정 전체에서 검증 정확도(Validation Accuracy)가 가장 높게 측정된 시점의 모델 가중치를 최종 모델로 저장하는 전략을 사용했다. 이 방식은 훈련 데이터에 과적합되지 않고, 실제 데이터와 가장 유사한 분포를 가진 검증 데이터에서 가장 우수한 일반화 성능을 보인 모델을 객관적으로 선택하기 위함이다.

```
def evaluate(model, dataloader, criterion, device):
    model.eval()
    running_loss = 0.0
    running_corrects = 0
    with torch.no_grad():
        progress_bar = tqdm(dataloader, desc="Validation")
        for inputs, labels in progress_bar:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)
            # 평가 시에는 원본 라벨로 손실을 계산해야 합니다.
            loss = criterion(outputs, torch.nn.functional.one_hot(labels, num_classes=num_classes).float())
            _, preds = torch.max(outputs, 1)
            running_loss += loss.item() * inputs.size(0)
            running_corrects += torch.sum(preds == labels.data)
    epoch_loss = running_loss / len(dataloader.dataset)
    epoch_acc = running_corrects.double() / len(dataloader.dataset)
    return epoch_loss, epoch_acc.item()
```

- **고급 정규화 전략:** 학습 과정에서 **CutMix**와 **MixUp** 데이터 증강 기법을 확률적으로 적용하여 과적합을 방지했다. 이 기법들은 두 개 이상의 이미지와 레이블을 혼합하여 새로운 학습 데이터를 생성함으로써, 모델이 이미지의 특정 영역에만 집중하지 않고 전체적인 맥락과 특징을 학습하도록 강제한다. 이를 통해 모델은 더 강건해지고 새로운 데이터에 대한 예측 능력이 향상되었다.

```python
def train_one_epoch(model, dataloader, criterion, optimizer, device):
    model.train()
    running_loss = 0.0

    progress_bar = tqdm(dataloader, desc="Training")
    for inputs, labels in progress_bar:
        inputs, labels = inputs.to(device), labels.to(device)

        # ⭐ CutMix 또는 Mixup 적용 (핵심 변경점)
        # 50% 확률로 CutMix나 Mixup 중 하나를 적용합니다.
        if torch.rand(1).item() < (CONFIG["cutmix_prob"] + CONFIG["mixup_prob"]):
            mixed_inputs, mixed_labels = cutmix_or_mixup(inputs, labels)
        else:
            mixed_inputs, mixed_labels = inputs, torch.nn.functional.one_hot(labels, num_classes=num_classes).float()

        optimizer.zero_grad()

        # 혼합된 이미지로 학습
        outputs = model(mixed_inputs)
        # 혼합된 라벨로 손실 계산
        loss = criterion(outputs, mixed_labels)

        loss.backward()
        optimizer.step()

        running_loss += loss.item() * inputs.size(0)
        progress_bar.set_postfix(loss=loss.item())

    epoch_loss = running_loss / len(dataloader.dataset)
    return epoch_loss
```

### 마) 모델 개선

- **TTA(Test-Time Augmentation)를 이용한 예측 성능 향상:** 최종 선정된 모델의 예측 안정성과 정확도를 더욱 높이기 위해, 추론 단계에서 **TTA(Test-Time Augmentation)** 기법을 적용했다. TTA는 하나의 테스트 이미지가 입력되었을 때, 원본 이미지뿐만 아니라 좌우 반전(Horizontal Flip)과 같은 여러 증강된 버전을 함께 생성하여 각각의 예측을 수행한다. 이후, 모든 버전의 예측 결과를 **종합(평균)하여 최종 진단 결과를 도출하는 방식**이다. 이러한 접근 방식은 촬영 각도나 방향에 따라 발생할 수 있는 모델의 미세한 오판 가능성을 줄여주며, 단일 예측에 비해 훨씬 더 강건하고 신뢰도 높은 결과를 제공한다.

```python
# --- 7. TTA를 사용한 예측 (⭐핵심⭐) ---
# TTA 라이브러리에서 사용할 변환 정의 (원본, 좌우반전)
tta_transforms = tta.Compose([
    tta.HorizontalFlip(),
])

# 학습된 모델을 TTA 모델로 감싸줍니다.
tta_model = tta.ClassificationTTAWrapper(model, tta_transforms)

print("\n--- Running Prediction with TTA ---")
# 검증 데이터셋에서 하나의 이미지를 가져와 테스트
sample_image, sample_label = image_datasets['val'][0]
sample_image = sample_image.unsqueeze(0).to(CONFIG["device"]) # 배치 형태로 변경

# TTA를 사용하여 예측
tta_model.eval() # 반드시 평가 모드로 설정
with torch.no_grad():
    # 일반 예측
    vanilla_output = model(sample_image)
    _, vanilla_pred = torch.max(vanilla_output, 1)

    # TTA 예측
    tta_output = tta_model(sample_image) # 증강된 이미지들의 예측 결과를 평균
    _, tta_pred = torch.max(tta_output, 1)

print(f"Sample Image True Label: {class_names[sample_label]}")
print(f"Standard Prediction: {class_names[vanilla_pred.item()]}")
print(f"TTA Prediction: {class_names[tta_pred.item()]}")
```

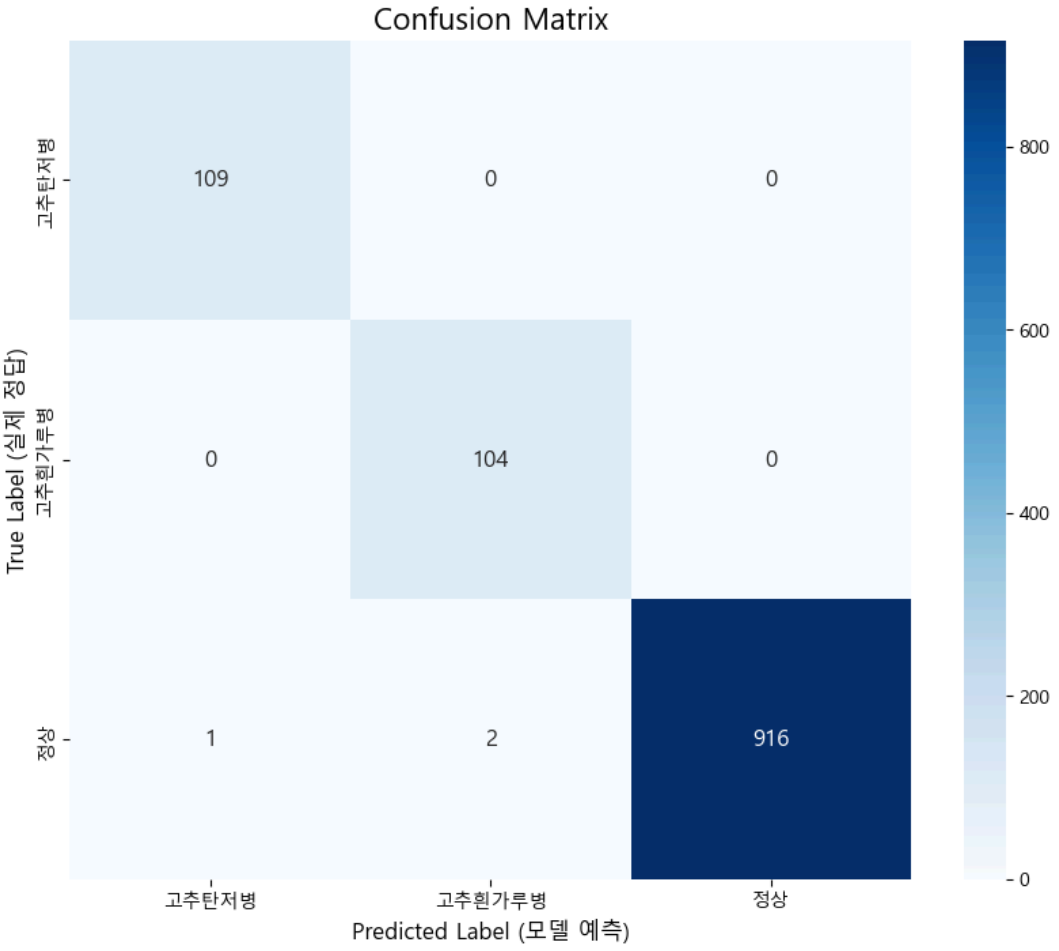
**바) 모델 평가**

- **테스트 데이터셋을 이용한 최종 성능 검증:** 학습된 모델 중 검증 데이터셋에서 가장 높은 성능을 보인 최적 모델을 선정하여, 학습에 사용되지 않은 1,132개의 테스트 이미지로 최종 성능을 평가했다. 평가 결과, **전체 정확도(Accuracy) 99.73%, 손실(Loss) 0.0064**라는 매우 높은 성능을 달성했다.
- **클래스별 상세 성능 분석 및 확장 가능성:** Classification Report를 통해 클래스별 성능을 정밀하게 분석한 결과, 모든 클래스에서 99% 이상의 높은 F1-Score를 기록했다. 특히 다음과 같은 성과가 돋보였다. **'정상' 클래스**는 정밀도(Precision) 100%를 달성하여, 질병에 걸린 작물을 정상으로 오진하는 경우가 없었다. **'고추탄저병'과 '고추흰가루병' 클래스**는 재현율(Recall) 100%를 기록하여, 실제 질병이 발생한 경우 이를 놓치지 않고 모두 정확하게 탐지해냈다. 이는 모델이 각기 다른 질병의 특징을 명확하게 구분하고 있으며, 실제 환경에서 특정 질병을 놓치거나 다른 질병으로 오진할 확률이 매우 낮음을 의미한다. **'고추' 모델의 성공적인 결과를 통해, 본 프로젝트에서 구축한 데이터 처리 및 모델링 방법론을 나머지 12종의 작물에 동일하게 적용할 경우에도 유사한 수준의 높은 진단 성능을 확보할 수 있을 것으로 판단되며, 이는 본 시스템의 높은 확장성과 실용성을 입증한다.**

## 📋 고추 병해 진단 모델 성능 리포트

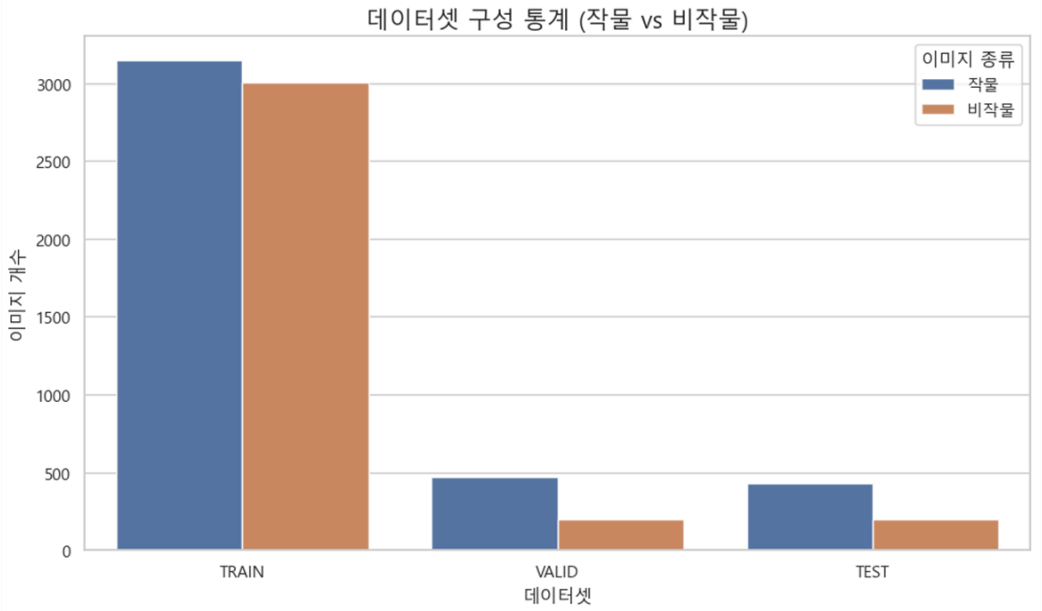
**Test Loss:** 0.0064 | **Test Accuracy:** 0.9973

| | Class | Precision | Recall | F1-score | Support |
|---|---|---|---|---|---|
| 1 | 고추탄저병 | 0.9909 | 1.0000 | 0.9954 | 109 |
| 2 | 고추흰가루병 | 0.9811 | 1.0000 | 0.9905 | 104 |
| 3 | 정상 | 1.0000 | 0.9967 | 0.9984 | 919 |
| 4 | | | | | |
| 5 | Accuracy | - | - | 0.9973 | 1132 |
| 6 | Macro avg | 0.9907 | 0.9989 | 0.9948 | 1132 |
| 7 | Weighted avg | 0.9974 | 0.9973 | 0.9974 | 1132 |



### 3) 작물 검증 모델

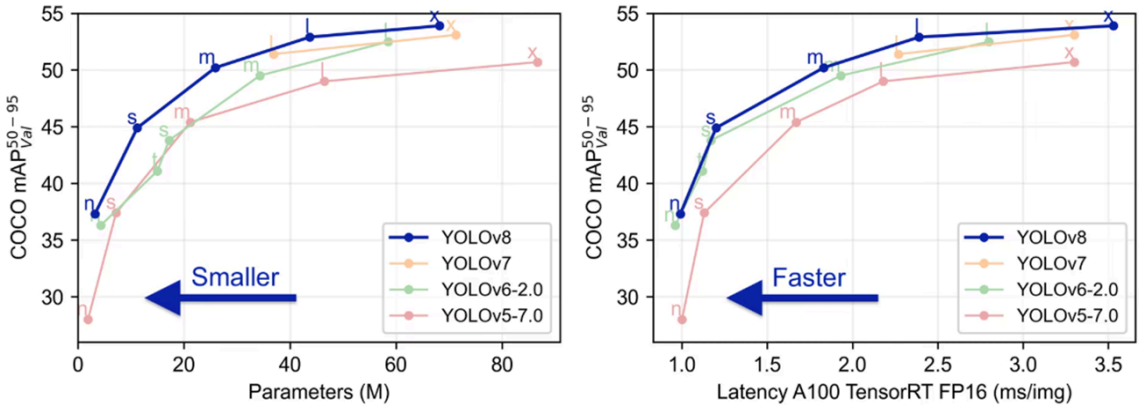
#### 가) 사전 정보 수집 및 전처리




- **이종 데이터셋을 활용한 데이터셋 구축:** 사용자가 업로드한 이미지가 실제 작물인지 아닌지를 판별하는 객체 탐지 모델을 개발하기 위해, 성격이 다른 두 종류의 데이터셋을 조합하여 학습 데이터를 구축했다. 먼저 다양한 종류의 작물 이미지를 확보하기 위해 공개 데이터셋 플랫폼인 **'YOLO Universe'에서 여러 작물에 바운딩 박스 라벨이 부착된 데이터셋들을 수집**했다. 총 4,043장의 작물 이미지를 확보하여 모델이 다양한 작물의 형태와 특징을 학습하도록 했다. 다음으로 작물이 아닌 일반 사물이나 풍경 이미지를 학습시키기 위해, 대규모 이미지 데이터셋인 **'COCO dataset'**의 일부를 활용했다. 총 3,406장의 이미지를 Negative 데이터로 사용하여 모델이 작물과 배경을 명확히 구분하는 능력을 갖추도록 했다.
- 라벨 표준화 및 데이터셋 구성: 수집한 Positive 데이터들은 각기 다른 작물 클래스 ID를 가지고 있었기 때문에, 이를 '작물'이라는 단일 클래스로 통합하는 전처리 과정을 수행했다. 모든 라벨 파일(.txt)을 순회하며 **각 바운딩 박스의 클래스 ID를 모두 '0'으로 통일**시켜, 모델이 특정 작물의 종류가 아닌 '작물의 일부(crop_part)' 자체를 탐지하도록 학습 목표를 설정했다. 그렇게 **전처리된 Positive 이미지와 Negative 이미지를 혼합**하여 최종적으로 학습(6,147장), 검증(671장), 테스트(631장) 데이터셋을 구성했다. data.yaml 파일을 통해 각 데이터셋의 경로와 단일 클래스명('crop_part')을 정의하여 YOLO 모델이 데이터를 인식하고 학습할 수 있도록 환경을 설정했다.

### 나) 모델 선정 및 개요

- **YOLOv8 모델 아키텍처 적용:** 사용자가 업로드한 이미지가 분석 가능한 작물 이미지가 맞는지 검증하는 단계에서는 실시간 객체 탐지(Real-time Object Detection)에 특화된 **YOLO(You Only Look Once)** 모델을 도입했다. YOLO는 이미지 전체를 한 번만 보고도 이미지 내 다수의 객체 위치와 종류를 동시에 예측하는 1-단계 탐지기(One-Stage Detector) 방식으로, 속도와 정확성 사이의 균형이 매우 뛰어나다. 작물 검증 모델은 사용자의 이미지를 1차 분류 및 2차 진단 모델로 전달하기 전에 유효성을 신속하게 판별해야 하므로, 실시간성이 중요한 이 과업에 YOLO가 가장 적합하다고 판단했다.
- **YOLOv8m 모델 선정 이유:** YOLOv8 시리즈는 다양한 크기의 모델(n, s, m, l, x)을 제공하여 사용 환경에 따라 성능과 속도를 조절할 수 있는 장점이 있다. 본 프로젝트에서는 중간 크기인 **'m(medium)' 모델**을 선택했다. 'm' 모델은 더 작은 'n(nano)'이나 's(small)' 모델보다 더 많은 파라미터를 가지고 있어 복잡하고 다양한 작물 이미지의 특징을 학습하는 데 유리하며, 더 큰 'l(large)'이나 'x(extra-large)' 모델보다는 가볍고 빨라 **모바일 앱 환경에서의 구동 부담이 적다**. 따라서, '텃밭닥터' 앱의 실시간성과 정확성을 모두 만족시키는 최적의 절충안으로 YOLOv8m을 최종 모델로 선정했다.




- **단일 클래스 탐지를 통한 범용성 확보:** 이 모델의 목적은 특정 작물의 종류를 구분하는 것이 아니라, 이미지가 '작물'인지 아닌지를 판별하는 것이다. 따라서 다양한 작물 데이터셋의 라벨을 모두 'crop_part'라는 단일 클래스로 통합하여 학습을 진행했다. 이를 통해 모델은 특정 작물에 국한되지 않고 다양한 종류의 작물이 가진 보편적인 시각적 특징을 학습하여, 새로운 종류의 작물 이미지에 대해서도 높은 탐지 성능을 보이는 범용적인 작물 검증 모델로 개발되었다.

### 다) 모델 분석 및 개선

- 모델 학습은 Ultralytics 프레임워크를 기반으로, COCO 데이터셋으로 사전 훈련된(pre-trained) YOLOv8m 가중치를 총 **100 Epoch**까지 추가 학습을 진행하는 **전이 학습(Transfer Learning)** 및 이어하기(Resume Training) 방식을 사용했다. 이미지 크기는 640x640, **배치 사이즈는 16**으로 설정하여 NVIDIA GeForce RTX 4070 GPU 환경에서 학습을 수행했다.
- **자동 최적화 및 데이터 증강:** 최적의 학습 파라미터를 찾기 위해 optimizer='auto' 설정을 활용했다. 이를 통해 프레임워크가 자동으로 데이터셋 특성에 맞는 **옵티마이저(AdamW), 학습률(0.002), 모멘텀(0.9)을 선정**하여 학습 효율을 높였다. 또한, YOLOv8에 내장된 **mosaic, fliplr, RandAugment, albumentations** 등 다채로운 데이터 증강 기법들이 학습 과정에 자동으로 적용되어, 모델이 다양한 환경 변화에 강건하게 작물을 탐지할 수 있도록 일반화 성능을 강화했다.

```python
def train_yolo_validator():
    """
    YOLOv8 모델을 사용하여 'crop_part'를 탐지하는
    유효성 검사 모델을 학습합니다.
    """
    # 1. yolov8m
    model = YOLO(r'C:\Users\427s2\KSEB\validation_\runs\train\crop_part_validator_v1\weights\last.pt')

    # 2. 모델 학습 실행
    # ultralytics의 기본 데이터 증강(augmentation) 기능이 자동으로 최적의 값으로 적용됩니다.
    # 따라서 대부분의 경우 별도 설정 없이도 좋은 성능을 기대할 수 있습니다.
    results = model.train(
        # --- 필수 설정 ---
        data=r'C:\Users\427s2\KSEB\validation_\new_dataset\data.yaml', # data.yaml 파일 경로
        epochs=100,                 # 전체 데이터셋 반복 학습 횟수
        imgsz=640,                  # 입력 이미지 크기

        # --- 성능 및 하드웨어 설정 ---
        batch=16,                   # 한 번에 처리할 이미지 수 (VRAM 부족 시 8, 4 등으로 줄이세요)
        device=0,                   # 사용할 GPU 번호 (0번 GPU를 사용)

        # --- 결과 저장 설정 ---
        project='runs/train',                   # 학습 결과가 저장될 상위 폴더
        name='crop_part_validator_v1',           # 이번 학습의 이름 (결과는 'runs/train/crop_part_validator_v1'에 저장됨)
        resume=True
    )
```

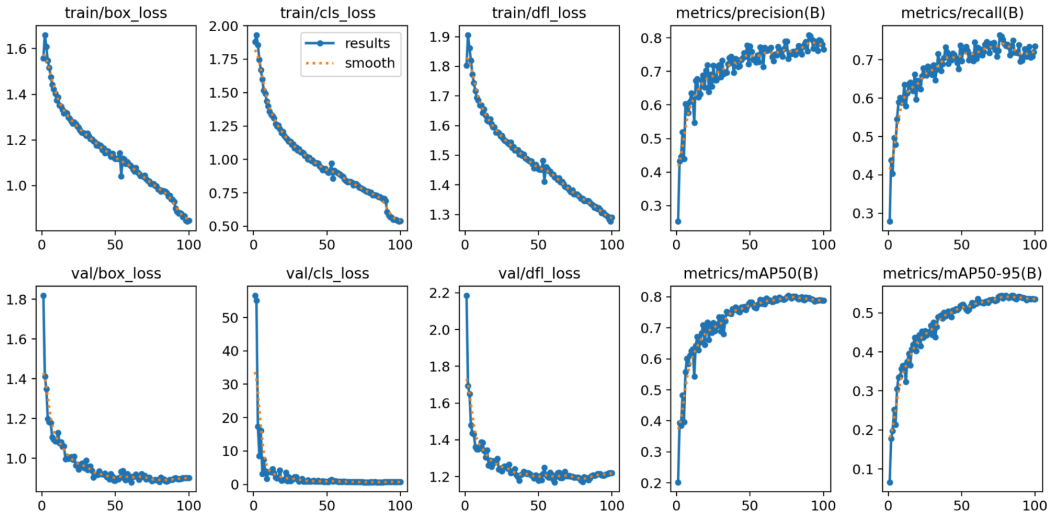
### 라) 모델 평가

- **테스트 데이터셋을 이용한 최종 성능 검증:** 학습이 완료된 모델의 최종 성능을 객관적으로 평가하기 위해, 학습 및 검증 과정에 사용되지 않은 631장의 테스트 이미지로 평가를 진행했다. 평가 결과, 모델은 **mAP50(Mean Average Precision at IoU 0.5) 82.9%, mAP50-95 54.9%** 라는 우수한 객체 탐지 성능을 기록했다.
- **성능 지표 분석:** mAP50 점수가 82.9%라는 것은 모델이 작물의 위치를 비교적 넉넉한 기준(IoU 0.5)으로 예측했을 때 **매우 정확하게 탐지함**을 의미한다. mAP50-95 점수는 더

엄격한 기준(IoU 0.5~0.95)에서의 평균 성능을 나타내며, 54.9%의 수치는 모델이 작물의 위치를 매우 정밀하게 예측하는 데에도 준수한 능력을 갖추고 있음을 보여준다. 또한, **76.5%의 정밀도(Precision)와 75.4%의 재현율(Recall)**은 모델이 작물이 아닌 객체를 작물로 잘못 판단하는 경우가 적고, 실제 작물 객체를 놓치지 않고 대부분 탐지해냄을 의미한다. 이러한 결과들은 본 모델이 사용자가 업로드한 이미지의 유효성을 판별하는 필터로서 충분한 신뢰성과 정확성을 갖추었음을 입증한다.

**YOLOv8 모델 성능 리포트**

| Class | Images | Instances | Precision | Recall | mAP@0.5 | mAP@0.5:0.95 |
|---|---|---|---|---|---|---|
| all | 631 | 1,648 | 0.765 | 0.754 | 0.829 | 0.549 |



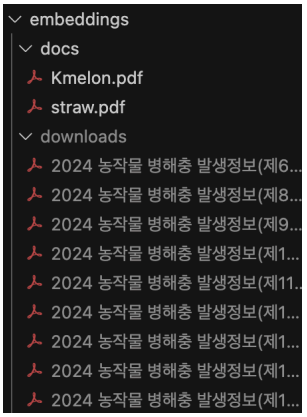

#### 4) 챗봇 모델

### 가) 사전 정보 수집 및 전처리

- 챗봇의 답변 정확성과 전문성을 확보하기 위해, 신뢰도가 높은 농업 관련 문서와 공공 데이터를 주요 정보원으로 활용했다. 수집된 데이터는 벡터 데이터베이스(Vector DB) 구축의 기반 자료로 사용된다.
- **주요 정보 수집원: 자체 수집 전문 문서(PDF)**와 농촌진흥청 **'농사로(NongSaRo)' Open API**가 사용된다. 먼저 자체 수집 전문 문서(PDF)는 특정 작물(예: 딸기, 참외)의 재배법, 병해충 정보 등 깊이 있는 지식을 담고 있는 전문 기술 문서이다. docs 폴더 내에 사전에 확보한 고품질의 PDF 파일을 저장하여, **PyPDFLoader**를 통해 텍스트 정보를 일괄적으로 추출했다. 이는 챗봇의 기본 지식 베이스를 형성한다. 다음으로 농촌진흥청 '농사로(NongSaRo)' Open API는 농촌진흥청에서 제공하는 최신 농업 기술 정보,

병해충 발생 정보 등 공신력 있는 데이터를 활용한다. get_nongsaro_documents 함수를 구현하여 API를 호출한다. "딸기", "참외", "단호박" 등 특정 키워드를 중심으로 관련 문서를 검색하고, 검색된 결과에서 **PDF 파일의 URL을 확보하여 자동으로 다운로드 및 텍스트를 추출**한다. 이로써 최신성과 전문성을 지속적으로 확보할 수 있다.

```python
def get_nongsaro_documents(keywords, year="2024"):
    docs = []
    url = "http://api.nongsaro.go.kr/service/dbyhsCccrrncInfo/dbyhsCccrrncInfoList"
    params = {"apiKey": NONGSARO_API_KEY, "sYear": year}
    res = requests.get(url, params=params)
    if res.status_code != 200:
        return docs
    root = ET.fromstring(res.content)
    items = root.findall(".//item")
    for item in items:
        title = item.findtext("cntntsSj")
        file_url = item.findtext("downFile")
        file_name = item.findtext("rtnOrginlFileNm")
        if file_url and file_name:
            path = download_file(file_url, file_name)
            if path:
                text = extract_text_from_pdf(path)
                for kw in keywords:
                    if kw in text:
                        docs.append(Document(
                            page_content=text,
                            metadata={"source": f"NongSaRo_{file_name}", "matched_keyword": kw}
                        ))
                        break
    return docs
```




- **데이터 수집 및 처리 과정:** docs 디렉토리에 저장된 PDF 파일들을 PyPDFLoader를 사용하여 로드하고, 각 페이지를 개별 Document 객체로 변환한다. 그 후, requests 라이브러리를 사용하여 농사로 API에 GET 요청을 보낸다. 응답으로 받은 XML 데이터에서 PDF 파일의 제목, 원본 파일명, 다운로드 URL 등의 정보를 파싱한다. 다운로드한 PDF 파일은 fitz (PyMuPDF) 라이브러리를 통해 텍스트 내용 전체를 추출한다. 이 과정에서 각 문서의 출처(source)와 검색 키워드(matched_keyword)를 메타데이터로 함께 저장하여 데이터의 출처를 명확히 했다. 마지막으로 로컬에서 로드한 문서와 API를 통해 수집한 문서를 하나의 리스트(all_docs)로 통합하여, 후속 단계인 텍스트 분할 및 임베딩을 위한 단일 데이터셋을 구축한다.

### 나) 모델 선정 및 개요

- '텃밭닥터' AI 챗봇은 사용자의 농업 관련 질문에 대해 다각적이고 정확한 답변을 제공하기 위해, 두 가지 핵심 AI 모델을 조합하여 설계되었다. 하나는 대화의 맥락을 이해하고 추론을 담당하는 대규모 언어 모델(LLM)이며, 다른 하나는 전문 지식 데이터베이스에서 사용자의 질문과 가장 관련성 높은 정보를 신속하게 찾아내는 **임베딩 모델**이다.
- **대규모 언어 모델 (LLM) OpenAI: gpt-3.5-turbo**는 OpenAI에서 개발한 대규모 언어 모델로, 뛰어난 자연어 이해 및 생성 능력, 그리고 비용 효율성을 바탕으로 본 프로젝트의 핵심 추론 엔진으로 채택되었다. 주요 역할로는 사용자의 질문 의도를 파악하고, 대화의 맥락을 유지하며, 복잡한 문제 해결을 위한 추론을 수행한다. 사용자의 질문이 주어지면, 사전에 정의된 도구들(벡터 DB 검색, 웹 검색, 직접 답변) 중 가장 적절한 것을 선택하고

실행 계획을 수립한다. 최종적으로 벡터 DB나 웹 검색을 통해 수집된 정보를 바탕으로, 사용자가 이해하기 쉬운 형태로 최종 답변을 종합하고 생성한다. 특히, 정보가 부족할 경우 '농업 전문가'로서의 역할을 부여받아(프롬프트 엔지니어링) 자체 지식을 활용해 신뢰도 높은 답변을 만들어낸다.

- **텍스트 임베딩 모델 HuggingFace sentence-transformers/all-MiniLM-L6-v2:** 이 모델은 문장과 단락을 고차원의 벡터 공간으로 변환하는 데 특화된 경량화 모델이다. 빠른 처리 속도와 높은 의미 검색 정확도를 제공하여 본 프로젝트의 다양한 텍스트 데이터를 벡터화하고 유사도 검색, 질의응답 및 맞춤형 답변 제공의 기반으로 활용된다.

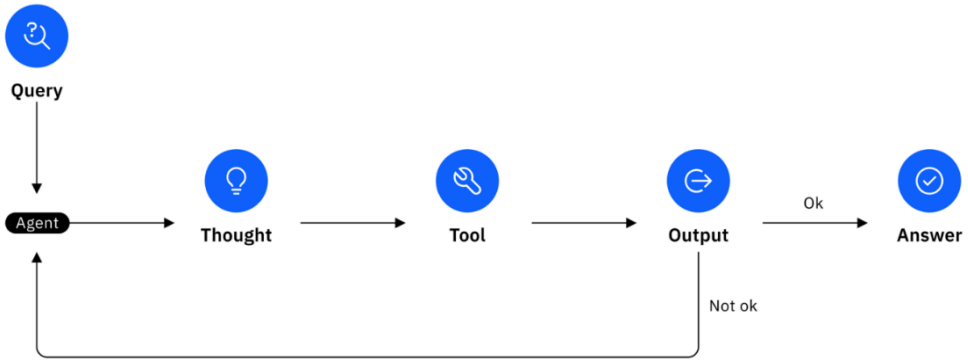
```
embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
vectorstore = FAISS.load_local("embeddings/faiss_index",
```

- **RAG(Retrieval-Augmented Generation, 검색 증강 생성)** 시스템의 핵심 요소로 선정되었다. 주요 역할로는 수집된 모든 농업 관련 전문 문서(PDF, API 데이터 등)를 의미를 함축한 벡터로 변환한다. 사용자의 질문이 입력되면, 이 모델은 질문 문장을 동일한 벡터 공간으로 변환한다. 그 후, FAISS 벡터 데이터베이스 내에서 **코사인 유사도**가 가장 높은 문서 조각(chunk)들을 신속하게 찾아낸다. 이 과정은 사용자의 질문 의도와 가장 관련성이 깊은 지식을 LLM에 전달하는 핵심적인 단계이다.

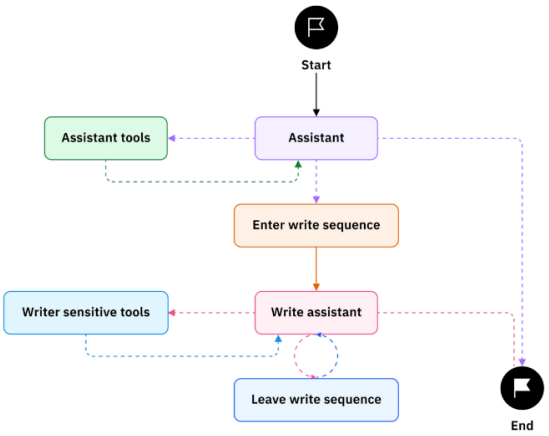
**다) 모델 분석 및 개선**

- '텃밭닥터'의 AI 챗봇은 단순히 사전 학습된 지식에만 의존하는 모델이 아니다. 사용자의 질문 의도를 명확히 파악하고, 주어진 상황에 가장 적합한 해결책을 동적으로 찾아내는 **자율적 에이전트(Autonomous Agent)** 아키텍처를 채택했다. 이 아키텍처의 핵심은 **LangGraph**와 **ReAct(Reason and Act)** 프레임워크를 기반으로, 여러 전문 도구(Tool)들을 지능적으로 선택하고 활용하는 데 있다.
- **ReAct 기반 에이전트 설계:** 사용자의 질문이 주어지면, LLM(gpt-3.5-turbo)은 먼저 문제 해결을 위한 계획을 수립한다. '이 질문에 답하기 위해 어떤 정보가 필요하며, 어떤 도구를 사용해야 가장 효과적일까?'를 스스로 판단한다. 수립한 계획에 따라, 가장 적합하다고 판단된 도구(Tool)를 실행한다. 예를 들어, 전문 지식이 필요하면 **전문지식 벡터 데이터베이스**를, 실시간 정보가 필요하면 **웹 서치**를 호출한다.

- 
```
tools = [direct_tool, knowledge_tool, web_tool]

# 3. LangChain ReAct 기반 Agent 구성
agent = initialize_agent(
    tools,
    llm,
    agent=AgentType.OPENAI_FUNCTIONS,  # ReAct 기반 에이전트
    verbose=True
)
```




- **LangGraph를 이용한 워크플로우 관리:** 에이전트의 복잡한 '생각과 행동' 과정을 체계적으로 관리하기 위해 **LangGraph**를 도입했다. LangGraph는 에이전트의 작업 흐름을 상태(State)를 가진 그래프(Graph) 형태로 정의한다. 이를 통해 각 단계에서 에이전트의 결정과 도구 사용 결과를 추적하고, 전체적인 대화의 흐름을 안정적으로 제어할 수 있다. 본 프로젝트에서는 사용자 입력부터 최종 답변 생성까지의 과정을 하나의 그래프로 구성하여, 에이전트의 작동을 명확하고 예측 가능하게 만들었다.




- **핵심 도구(Tools) 분석:** 에이전트는 사용자의 질문 유형에 따라 세 가지 핵심 도구를 전략적으로 선택하여 사용한다. **첫번째로 knowledge_db (전문 지식 검색 도구)의 경우**, 사용자의 질문과 의미적으로 가장 유사한 정보를 자체 구축한 농업 전문 지식 벡터 데이터베이스(FAISS)에서 찾아낸다. 작물 재배법, 병해충 진단 및 방제 등 대부분의 전문적인 농업 질문에 대한 답변의 근거를 마련하는 핵심 도구이다.

```
# knowledge_db 툴
knowledge_tool = Tool(
    name="knowledge_db",
    func=knowledge_with_fallback,
    description=(
        "전문 PDF 문서와 농사로 데이터에서 관련 정보를 찾아 상세히 요약합니다. "
        "벡터 DB에 충분한 정보가 없으면 농업 전문 AI 지식으로 자동 보완합니다. "
        "작물 재배, 병해충 방제, 농업 기술 등 모든 농업 관련 질문에 대응합니다. "
        "예: '딸기 잎에 흰 반점 생겼어', '토마토 재배법', '작물 병해충 원인과 대응 방
법'" )
)
```

- 만약 벡터 DB에서 충분한 정보를 찾지 못했을 경우(예: 검색 결과가 없거나 내용이 부실할 때), 에이전트는 여기에 머무르지 않는다. 즉시 '농업 전문가' 역할을 부여받은 LLM을 통해 직접 답변을 생성하는 **Fallback(대체 작동)** 메커니즘이 발동된다. 이는 정보 부족으로 인한 답변 실패를 방지하고, 어떠한 상황에서도 사용자에게 유용한 정보를 제공하기 위한 안정장치이다.

```
def knowledge_with_fallback(query: str) -> str:
    """벡터 DB 검색 후 결과가 부족하면 GPT로 fallback"""
    try:
        response = knowledge_chain.run(query)
        fallback_phrases = [
            "찾지 못했습니다",
            ...생략
        ]

        # 불충분한 답변이면 GPT로 직접 답변
        if any(phrase in response for phrase in fallback_phrases) or len(response.strip()) < 50:
            print(f"[fallback] knowledge_db 응답 부족 → direct_answer 재시도: {query}")
            # 농업 전문 프롬프트로 GPT 답변 생성
            agricultural_prompt = f"""[농업 전문 프롬프트 내용] """
            gpt_result = direct_chain.invoke({"query": agricultural_prompt})
            final_answer = gpt_result.content if hasattr(gpt_result, 'content') else
str(gpt_result)
            return response
```

**두번째로 web_search (실시간 정보 검색 도구)의 경우,** Tavily 검색 엔진 API를 활용하여 웹상의 최신 정보를 실시간으로 검색한다. 예를 들어, "오늘 수원 날씨 어때?", "이번 주말 강수량 예보 알려줘" 와 같이 벡터 DB에는 저장되어 있지 않은 실시간성 정보나 최신 뉴스에 대응하기 위해 사용된다.

```
tavily = TavilySearch(api_key=TAVILY_API_KEY)
web_tool = Tool(name="web_search", func=tavily.run, description=
                "실시간 정보(날씨, 현재 기온, 뉴스 등)를 검색합니다. "
                "예: '오늘 날씨 어때?', '지금 기온이 몇 도야?', '경기도 지역 강수량 예보 알려
```

**마지막으로 direct_answer (직접 답변 도구)**는 간단한 상식이나 일반적인 지식에 대한 질문에 대해, 별도의 검색 과정 없이 LLM이 직접 답변을 생성한다. 예를 들어, "작물의 광합성 조건은?", "딸기는 채소야 과일이야?" 와 같은 질문에 신속하게 답변하여 효율성을 높인다.

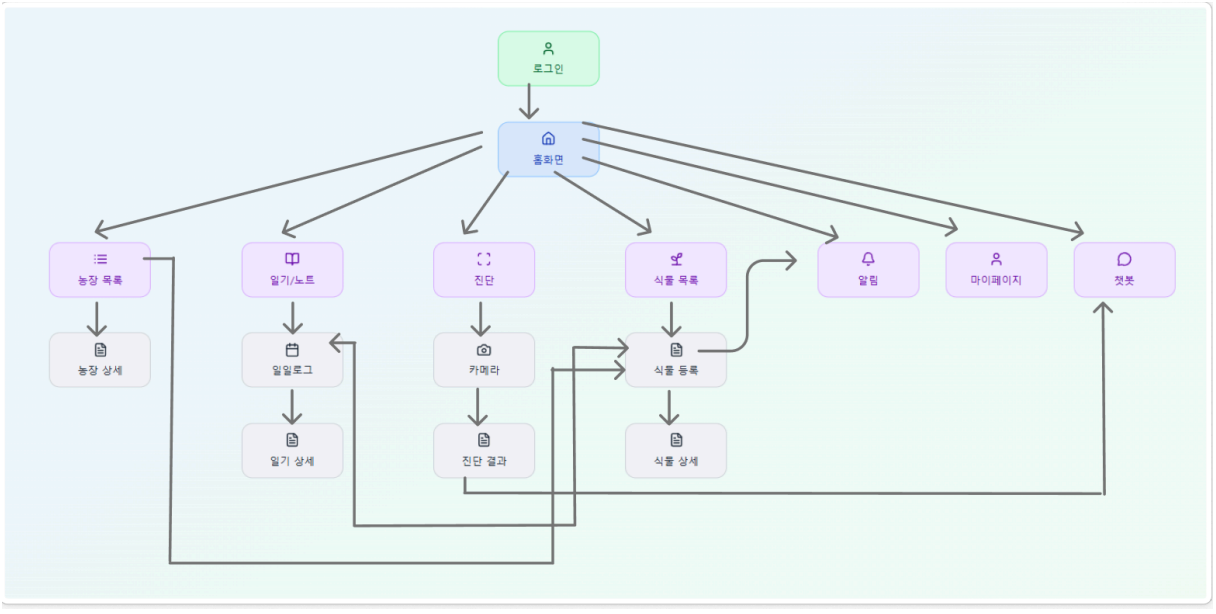
```
llm = ChatOpenAI(model_name="gpt-3.5-turbo", openai_api_key=OPENAI_API_KEY)
print(f"[LangGraph Agent] 사용 모델: {llm.model_name}")

# 2. 툴 정의
# direct_answer 툴 (gpt 직접 응답)
direct_prompt = PromptTemplate(input_variables=["query"], template="{query}")
direct_chain = direct_prompt | llm  # RunnableSequence

direct_tool = Tool(
    name="direct_answer",
    func=lambda q: direct_chain.invoke({"query": q}),
    description="작물 재배나 상식 질문에 대해 직접 응답합니다. 예: '딸기 수확 시기 알려줘', '작물의 광합성 조건은?'"
)
```

**대화 맥락 유지:** 챗봇은 세션(Session) 기반으로 이전 대화 기록을 모두 기억한다. 사용자가 새로운 질문을 할 때마다, 에이전트는 **이전 대화의 맥락 전체(conversation_context)를 함께 전달**받는다. 이를 통해 "그럼 그 약은 어디서 살 수 있어?" 와 같은 후속 질문의 의도를 정확히 파악하고, 단절되지 않는 자연스러운 대화를 이어갈 수 있다.

```python
# AI 응답 생성
try:
    # 기존 대화 히스토리 가져오기
    previous_messages = db.query(database.ChatMessage).filter(
        database.ChatMessage.session_id == session.id
    ).order_by(database.ChatMessage.timestamp.asc()).all()

    # 대화 히스토리 포함하여 AI 에이전트 실행
    conversation_context = ""
    for msg in previous_messages:
        if msg.role == "user":
            conversation_context += f"사용자: {msg.query}\n"
        else:
            conversation_context += f"AI: {msg.query}\n"

    # 현재 질문과 함께 컨텍스트 전달
    full_query = f"이전 대화:\n{conversation_context}\n\n현재 질문: {request.query}"
    ai_response = await run_agent(full_query)

    # AI 응답 메시지 저장
    assistant_message = database.ChatMessage(
        session_id=session.id,
        role="assistant",
        query=ai_response
    )
    db.add(assistant_message)
```

### 3.2.5 프론트엔드

#### 1) 프론트엔드 - 화면 구조도




본 서비스의 프론트엔드는 Flutter 기반으로 개발되었으며, 로그인 이후 홈 화면 (HomeScreen)을 중심으로 하단 네비게이션 메뉴를 통해 각 기능 모듈로 이동할 수 있다. 각 모듈은 독립적인 화면 세트를 포함하며, 공통적으로 JWT 인증 기반 API 연동, 애니메이션 전환, 로컬 토큰 저장(SharedPreferences), 에러 처리 및 로딩 UI를 제공한다.

### 2) 핵심 모듈 기능

#### (1) 로그인/회원가입 모듈

- **기능**: 사용자 인증 및 신규 계정 생성.
- **구성 화면**:
    - 로그인 화면: 이메일/비밀번호 입력, 서버 인증 요청(POST/api/auth/login).
    - 회원가입 화면: 이메일·비밀번호·닉네임 입력 후 계정 생성(POST /api/auth/register).
    - 소셜 로그인: Kakao OAuth2 인증 지원.
- **특징**: JWT 기반 인증 토큰 발급 및 로컬 저장, 인증 성공 시 홈 화면으로 이동.

#### (2) 챗봇 모듈

- **기능**: AI 기반 상담 및 질의응답.
- **구성 화면**:
    - **챗봇 대화**: 세션별 메시지 히스토리 관리, 질의 전송 → 서버 응답 표시.
- **특징**: 서버 응답 포맷(A/B) 대응 로직 구현, 새 세션 자동 생성, 오류 시 사용자 알림 처리.

#### (3) 진단 모듈

- **기능**: 식물 사진을 기반으로 한 AI 질병/상태 분석.
- **구성 화면**:
    - **이미지 업로드**: 카메라 촬영·갤러리 선택 → 서버 분석 요청(POST/api/photo-analysis/analyze).
    - **진단 결과**: 분석 이미지, 요약, 병명 및 해결책 표시, 챗봇 상담으로 연계.
- **특징**: 촬영 이미지 회전 보정, 업로드 중 로딩 오버레이, 분석 실패 시 에러 메시지 제공.

#### (4) 식물 관리 모듈

- **기능**: 사용자가 키우는 식물의 등록·조회·상세 관리.
- **구성 화면**:
  **식물 목록**: 카드형 리스트로 사용자 식물 표시, 알림 주기(물·가지치기·영양제) 요약.
    - **식물 등록**: 멀티스텝 마법사 방식(식물 선택 → 별명 → 장소 → 관리 주기 설정 → 메모·사진).
    - **상세 화면**: 개별 식물 이미지, 별명, 장소 확인 및 수정/삭제.
- **특징**: 서버 연동(GET/POST/api/user-plants), 위치 기반 농장 선택 연계, 애니메이션 적용.

**(5) 일지 모듈**

- **기능**: 날짜별 식물 관리 활동 기록 및 조회.
- **구성 화면**:
    - **달력형 일지**: TableCalendar 기반, 식물별 필터 지원.
    - **상세 일지**: 기록된 사진·메모·관리 정보 확인, 수정 및 삭제 가능.
- **특징**: 서버 연동(/api/diaries), 수정 시 노트 작성 화면과 연계.

**(6) 농장 탐색 모듈**

- **기능**: 공공/주변 텃밭 검색 및 선택.
- **구성 화면**:
    - **농장 목록**: 키워드 기반 검색 결과 제공.
    - **주변 탐색**: 위치 기반 근처 농장 조회.
- **특징**: 검색 결과는 식물 등록 단계의 “장소 선택”과 연동됨.

**(7) 노트 모듈**

- **기능**: 사진 및 텍스트 기반의 관리 노트 작성.
- **구성 화면**:
    - **노트 작성/수정**: 식물 선택, 사진 첨부, 물주기/비료/가지치기 체크.
    - **노트 상세**: 작성 내용 열람 및 수정.
- **특징**: /api/diaries API 재사용, 이미지 업로드 포함.

**(8) 알림 모듈**

- **기능**: 관리 주기 알림 설정 및 알림 메시지 확인.
- **구성 화면**:
    - **알림 설정**: 물주기/비료/가지치기 주기(일 단위) 선택 및 알림 on/off.
    - **알림 확인**: 알림 목록 표시.
- **특징**: 설정 값은 식물 모델에 반영되어 목록 화면 칩으로 표시.

**(9) 사용자 관리 모듈 (마이페이지)**

- **기능**: 계정 및 프로필 관리.
- **구성 화면**:
    - **마이페이지**: 프로필 조회·닉네임/이미지 변경.

    - **설정 화면**: 로그아웃, 회원 탈퇴.
- **특징**: PATCH /auth/me/nickname, PATCH /auth/me/profile-image, DELETE /auth/me API와 연계.

### 3.2.6 백엔드

#### 1) 주요 모듈 설계 및 서비스

본 '텃밭 작물 관리 서비스'는 사용자 관리 및 인증, 텃밭 정보 관리, 작물 정보 관리, AI를 활용한 진단 및 상담, 작물 일지 기록 등 다양한 기능을 제공하기 위해 여러 독립적인 모듈로 구성된다. 각 모듈의 상세 설계와 제공 서비스는 다음과 같다.

#### 가) 사용자 인증 및 프로필 관리 모듈 (Authentication & Profile Module)

- **(1) 모듈 개요**
    - 사용자의 계정 생성, 시스템 접근 인증(로그인/로그아웃), JWT 토큰 기반의 인증 상태 관리, 사용자 정보 조회 및 수정을 담당하는 핵심 모듈이다.
- **(2) 주요 기능**
    - 이메일 기반 회원가입 및 사용자 계정 생성
    - 이메일과 비밀번호를 이용한 로그인 및 JWT 토큰 발급
    - 토큰 무효화를 통한 안전한 로그아웃
    - Refresh Token을 이용한 Access Token 재발급
    - 사용자 프로필(닉네임, 프로필 사진 등) 조회 및 수정
    - 사용자 계정 삭제(회원 탈퇴)
- **(3) 모듈 설계**
    - 회원가입 시, 이메일 중복 검사와 데이터 유효성 검증을 거친 후, 비밀번호를 해시 암호화하여 데이터베이스에 저장한다.
    - 로그인 성공 시, JWT Access Token과 Refresh Token을 생성하여 발급하고, 로그아웃 시에는 해당 토큰을 블랙리스트에 추가하여 무효화 처리한다.
    - 모든 사용자 정보 관련 요청은 JWT 토큰 검증을 통해 인가된 사용자인지 확인한 후에 처리된다.

- **(4) 제공 서비스 (API)**
    - **POST /auth/register**: 사용자 회원가입
    - **POST /auth/login**: 사용자 로그인

- **POST /auth/logout**: 사용자 로그아웃
- **POST /auth/token/refresh**: JWT 토큰 재발급
- **GET /auth/me**: 내 정보 조회
- **PATCH /auth/me/profile-image/upload**: 사용자 프로필 이미지 변경
- **PATCH /auth/me/nickname**: 닉네임 변경
- **DELETE /auth/me**: 회원 탈퇴

**나) 텃밭 정보 관리 모듈 (Farm Information Module)**

- **(1) 모듈 개요**
  - 관리자가 공공 텃밭 정보를 시스템에 등록, 수정, 삭제하며 사용자는 이 정보를 조회하고 검색할 수 있도록 하는 모듈이다.
- **(2) 주요 기능**
  - 관리자의 신규 텃밭 정보(기본 정보, 위치, 부대시설 등) 등록
  - 전체 텃밭 목록 및 특정 텃밭의 상세 정보 조회
  - 키워드(이름, 주소 등)를 통한 텃밭 검색
  - 사용자 현재 위치 기반 주변 텃밭 조회
- **(3) 모듈 설계**
  - 텃밭 정보는 관리자 권한을 확인한 후 데이터베이스에 저장/수정/삭제된다.
  - 주변 텃밭 조회 시, 사용자의 위도/경도 정보를 받아 반경 내의 텃밭을 검색하고 거리를 계산하여 가까운 순으로 정렬하여 지도로 제공한다.
- **(4) 제공 서비스 요약 (API)**
  - **POST /admin/farms**: (관리자) 새로운 텃밭 등록
  - **GET /api/farms**: 전체 텃밭 목록 조회
  - **GET /api/farms/{farmId}**: 특정 텃밭 정보 조회
  - **GET /api/farms/search**: 텃밭 목록 검색
  - **GET /api/farms/nearby**: 주변 텃밭 정보 조회

**다) 식물 정보 관리 모듈 (Plant Information Module)**

- **(1) 모듈 개요**
  - 서비스에서 제공하는 모든 식물의 기본 정보를 관리하고, 사용자가 자신이 키우는 식물을 등록하여 관리할 수 있도록 지원하는 모듈이다. 이 모듈은 관리자가 관리하는 **공통 식물 정보**와 **사용자별 식물 정보**로 나뉜다.
- **(2) 주요 기능**

    - (관리자) 공통 식물 정보(이름, 분류 등) 등록, 수정, 삭제
    - (사용자) 자신이 키우는 식물을 별명, 심은 날짜 등과 함께 개별적으로 등록 및 관리
    - 사용자가 등록한 자신의 식물 목록 및 상세 정보 조회, 수정, 삭제
- **(3) 모듈 설계**
    - 관리자는 모든 사용자가 조회할 수 있는 공통 식물 데이터를 관리한다.
    - 사용자는 이 공통 식물 정보를 바탕으로 자신의 '사용자 식물'을 생성하며, 이는 사용자 계정에 종속되어 관리된다. 사용자 식물 등록 시 초기 알림 일정이 설정될 수 있다.
- **(4) 제공 서비스 요약 (API)**
    - **POST /admin/plants**: (관리자) 새로운 공통 식물 등록
    - **GET /api/plants**: 전체 공통 식물 목록 조회
    - **GET /api/plants/{plantId}**: 특정 공통 식물 정보 조회
    - **POST /api/user-plants**: 사용자 식물 정보 등록
    - **GET /api/user-plants**: 내 식물 목록 조회
    - **GET /api/user-plants/{userPlantId}**: 특정 내 식물 정보 조회
    - **PUT /api/user-plants/{userPlantId}**: 특정 내 식물 정보 수정

**라) AI 진단 및 상담 모듈 (AI Diagnosis & Chatbot Module)**

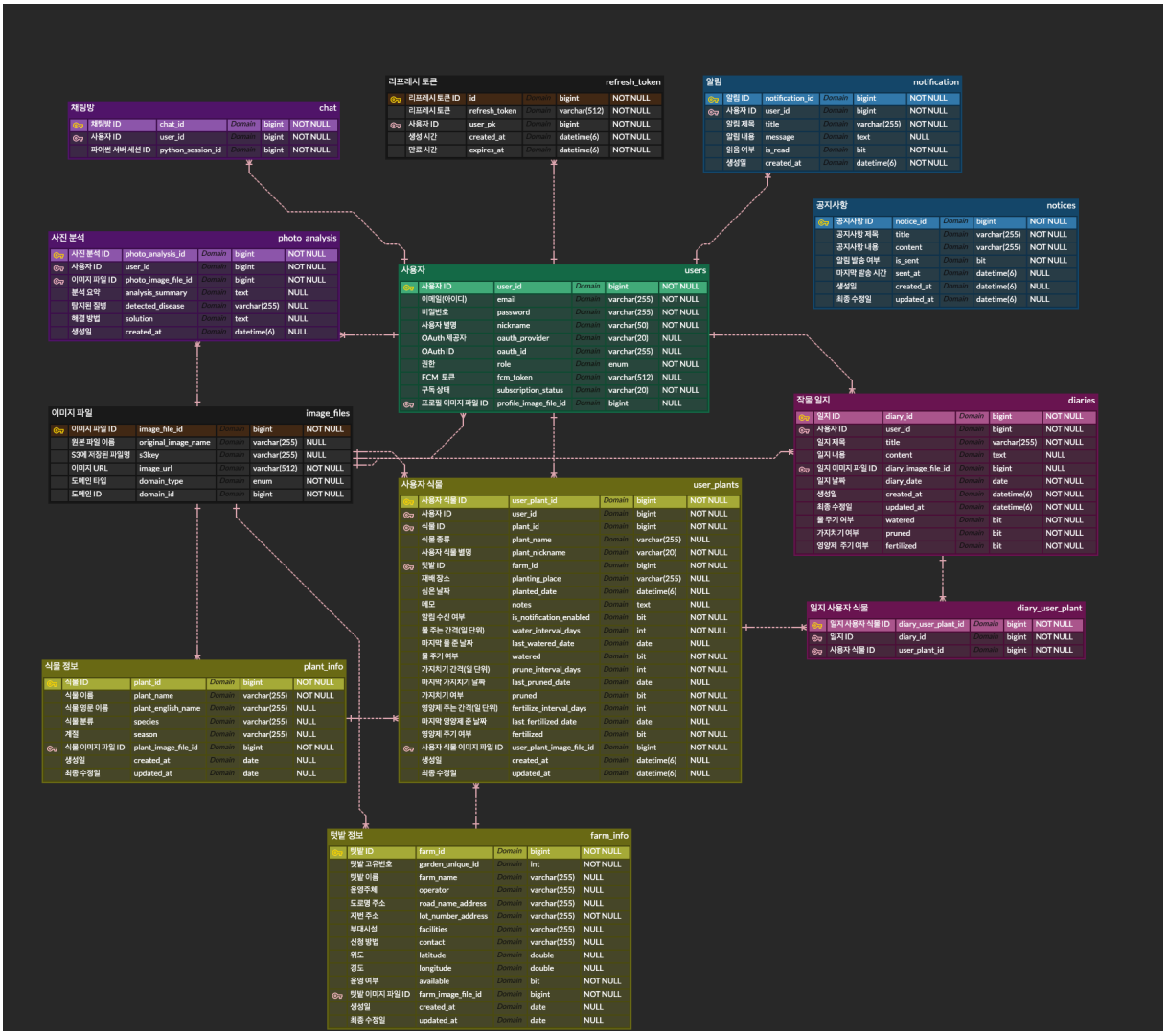
- **(1) 모듈 개요**
    - AI 기술을 활용하여 사용자가 업로드한 작물 사진을 분석하고, 챗봇을 통해 작물 관련 질문에 대한 맞춤형 답변을 제공하는 서비스의 핵심 지능형 모듈이다.
- **(2) 주요 기능**
    - 식물 사진 AI 분석을 통한 질병, 해충, 영양 상태 진단
    - 분석 결과에 따른 맞춤형 치료법 및 예방 방법 추천
    - 과거 사진 분석 이력 조회 및 관리
    - 사용자 질문을 이해하고 답변하는 AI 챗봇 기능
    - 챗봇 대화 이력 저장, 검색 기능
- **(3) 모듈 설계**
    - 사용자가 이미지를 업로드하면, 이미지를 전처리한 후 작물별 특화 AI 모델을 호출하여 분석을 수행한다. 분석 결과는 신뢰도 점수와 함께 데이터베이스에 저장된다.
    - 챗봇은 사용자의 질문 텍스트와 등록된 식물(UserPlant) 정보를 함께 분석하여 AI 모델을 통해 최적화된 답변을 생성하고, 대화 기록을 저장한다.
- **(4) 제공 서비스 (API)**
    - **POST /api/photo-analysis/analyze**: AI 이미지 분석 및 치료법 추천

- **POST /api/chat**: 챗봇 질문 전송
- **GET /api/chat/history/messages/all**: 특정 대화의 전체 응답 조회
- **GET /api/chat/sessions**: 최신 챗봇 대화방 목록 조회

**마) 재배 일지 및 알림 모듈 (Diary & Notification Module)**

- **(1) 모듈 개요**
  - 사용자가 등록한 식물에 대한 관리 및 활동 내역을 날짜별로 기록(재배 일지)하고, 물주기 등 필요한 작업에 대한 알림을 받을 수 있도록 하는 사용자 활동 지원 모듈이다.
- **(2) 주요 기능**
  - 날씨, 작업 종류, 내용, 사진 등을 포함한 재배 일지 작성, 조회, 수정, 삭제
  - 기간별 또는 특정 식물별로 일지 검색 및 필터링
  - 물주기, 시스템 공지 등 FCM 푸시 알림 발송
  - 사용자의 알림 목록 조회, 읽음 처리, 삭제
  - 읽지 않은 알림 개수 조회
- **(3) 모듈 설계**
  - 재배 일지 작성 시 사용자 인증 및 식물 소유권을 확인한 후 데이터베이스에 저장한다.
  - 알림은 관리자가 생성하거나 시스템에서 특정 조건(예: 물주기 예약 시간)에 따라 트리거되어 해당 사용자의 FCM 토큰을 통해 푸시 알림으로 발송된다.
- **(4) 제공 서비스 (API)**
  - **POST /api/diaries**: 새 일지 생성
  - **GET /api/diaries/{diaryId}**: 특정 일지 조회
  - **GET /api/diaries/my-diaries**: 내 모든 일지 최신순 조회
  - **POST /api/notifications/{userId}**: (관리자) 사용자 알림 생성 및 발송
  - **GET /api/notifications**: 내 알림 목록 조회
  - **PATCH /api/notifications/{notificationId}/read**: 단일 알림 읽음 처리

## 2) ERD 설계




https://www.erdcloud.com/d/9qoH9YJ2N85SbdsP2

# 4. 부록

## 4.1 모델 성능 비교표

(가) 작물 병해 분류 모델(예시 '고추' 모델)

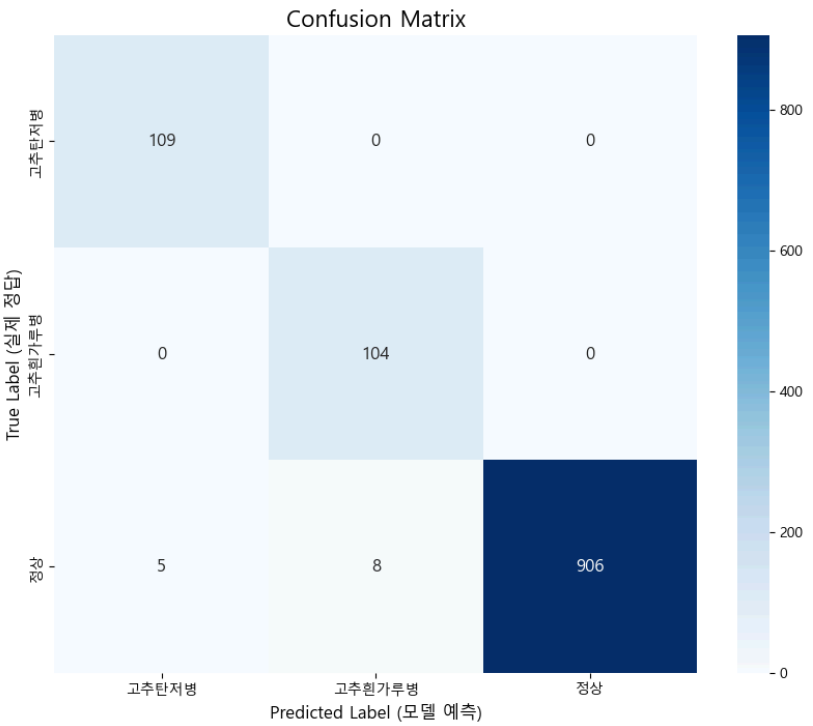
- EfficientNet-B1 모델

### 📋 고추 병해 진단 모델 성능 리포트

**Test Accuracy:** 0.9885

| | Class | Precision | Recall | F1-score | Support |
|---|---|---|---|---|---|
| 1 | 고추탄저병 | 0.9561 | 1.0000 | 0.9776 | 109 |
| 2 | 고추흰가루병 | 0.9286 | 1.0000 | 0.9630 | 104 |
| 3 | 정상 | 1.0000 | 0.9859 | 0.9929 | 919 |
| 4 | | | | | |
| 5 | Accuracy | - | - | 0.9885 | 1132 |
| 6 | Macro avg | 0.9616 | 0.9953 | 0.9778 | 1132 |
| 7 | Weighted avg | 0.9892 | 0.9885 | 0.9887 | 1132 |

-

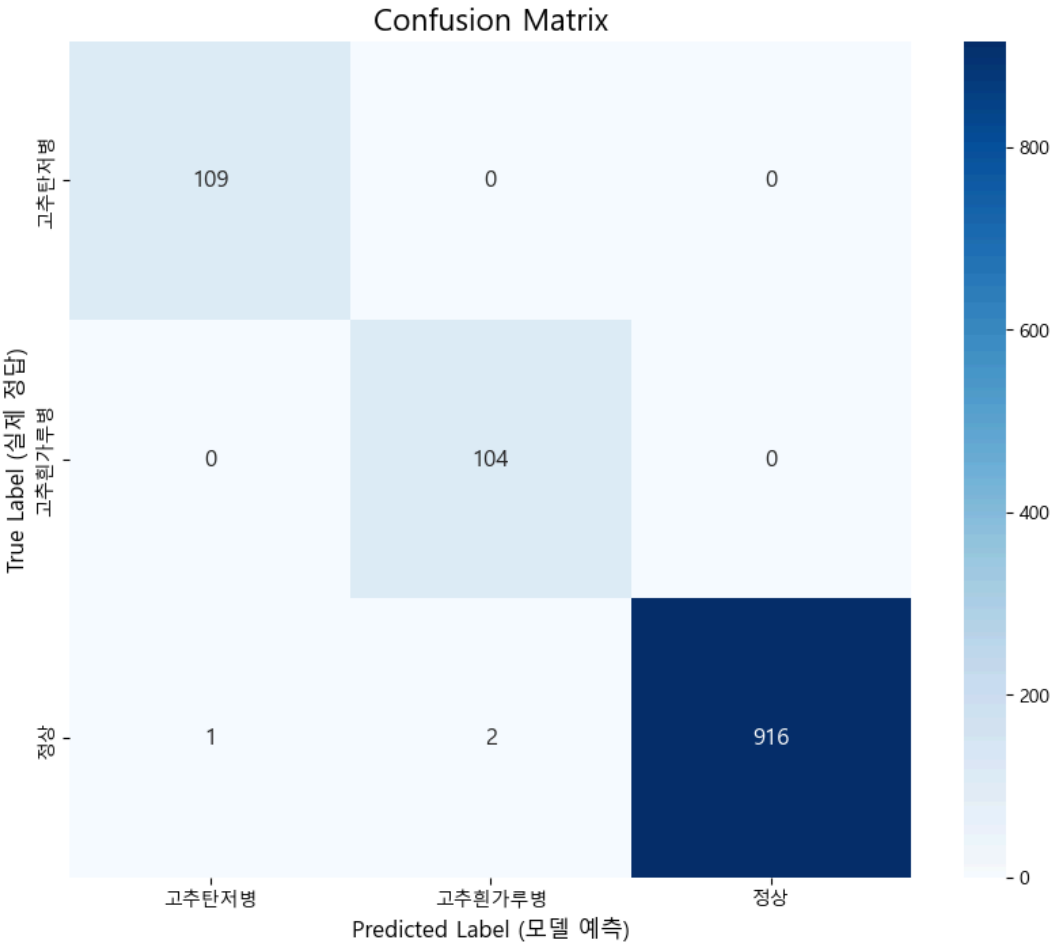


- ConvNeXt-tiny모델

## 📋 고추 병해 진단 모델 성능 리포트

**Test Loss:** 0.0064 | **Test Accuracy:** 0.9973

| | Class | Precision | Recall | F1-score | Support |
|---|---|---|---|---|---|
| 1 | 고추탄저병 | 0.9909 | 1.0000 | 0.9954 | 109 |
| 2 | 고추흰가루병 | 0.9811 | 1.0000 | 0.9905 | 104 |
| 3 | 정상 | 1.0000 | 0.9967 | 0.9984 | 919 |
| 4 | | | | | |
| 5 | Accuracy | - | - | 0.9973 | 1132 |
| 6 | Macro avg | 0.9907 | 0.9989 | 0.9948 | 1132 |
| 7 | Weighted avg | 0.9974 | 0.9973 | 0.9974 | 1132 |



## 4.2 코드 스냅샷(중요 로직 캡쳐)

### 1) ML/DL

```
# --- 클래스 가중치 계산 및 적용 ---
print("클래스 가중치 계산을 시작합니다...")

# 1. 훈련 데이터셋의 클래스별 데이터 수 계산
# train_dataset.targets는 각 이미지의 레이블(0, 1, 2...)을 담고 있는 리스트입니다.
class_counts = Counter(train_dataset.targets)
class_counts = [class_counts[i] for i in range(len(train_dataset.classes))] # 클래스 인덱스 순으로 정렬

print("\n훈련 데이터셋의 클래스별 데이터 수:")
for i, count in enumerate(class_counts):
    print(f"- {train_dataset.classes[i]}: {count}장")


num_total_samples = len(train_dataset)
weights = [num_total_samples / count for count in class_counts]

weights_tensor = torch.FloatTensor(weights)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
weights_tensor = weights_tensor.to(device)
print(f"\n현재 사용 중인 디바이스: {device}")

criterion = torch.nn.CrossEntropyLoss(weight=weights_tensor)

print("\n계산된 클래스 가중치:")
print(weights_tensor)
print("\nCrossEntropyLoss에 가중치가 성공적으로 적용되었습니다.")
print("이제부터 모델 학습 시 이 'criterion' 변수를 손실 함수로 사용하세요.")
```

*# 클래스 가중치 계산 및 적용*

```python
class ApplyCLAHE:
    def __init__(self, clip_limit=2.0, tile_grid_size=(8, 8)):
        self.clip_limit = clip_limit
        self.tile_grid_size = tile_grid_size

    def __call__(self, pil_img):
        clahe = cv2.createCLAHE(clipLimit=self.clip_limit, tileGridSize=self.tile_grid_size)
        cv_img = cv2.cvtColor(np.array(pil_img), cv2.COLOR_RGB2BGR)
        lab_img = cv2.cvtColor(cv_img, cv2.COLOR_BGR2LAB)
        l, a, b = cv2.split(lab_img)
        l_clahe = clahe.apply(l)
        lab_img_clahe = cv2.merge((l_clahe, a, b))
        bgr_img_clahe = cv2.cvtColor(lab_img_clahe, cv2.COLOR_LAB2BGR)
        return Image.fromarray(cv2.cvtColor(bgr_img_clahe, cv2.COLOR_BGR2RGB))

# ApplyGaussianBlur는 보통 문제가 없지만, 만약 비슷한 오류가 발생한다면
# 이 클래스도 위와 동일한 방식으로 수정할 수 있습니다.
class ApplyGaussianBlur:
    """
    PIL 이미지를 입력받아 가우시안 블러를 적용하여 노이즈를 제거합니다.
    """
    def __init__(self, kernel_size=(5, 5), sigma_x=0):
        self.kernel_size = kernel_size
        self.sigma_x = sigma_x

    def __call__(self, pil_img):
        cv_img = cv2.cvtColor(np.array(pil_img), cv2.COLOR_RGB2BGR)
        blurred_img = cv2.GaussianBlur(cv_img, self.kernel_size, self.sigma_x)
        return Image.fromarray(cv2.cvtColor(blurred_img, cv2.COLOR_BGR2RGB))
```

```python
# --- 7. TTA를 사용한 예측 (⭐핵심⭐) ---
# TTA 라이브러리에서 사용할 변환 정의 (원본, 좌우반전)
tta_transforms = tta.Compose([
    tta.HorizontalFlip(),
])

# 학습된 모델을 TTA 모델로 감싸줍니다.
tta_model = tta.ClassificationTTAWrapper(model, tta_transforms)

print("\n--- Running Prediction with TTA ---")
# 검증 데이터셋에서 하나의 이미지를 가져와 테스트
sample_image, sample_label = image_datasets['val'][0]
sample_image = sample_image.unsqueeze(0).to(CONFIG["device"]) # 배치 형태로 변경

# TTA를 사용하여 예측
tta_model.eval() # 반드시 평가 모드로 설정
with torch.no_grad():
    # 일반 예측
    vanilla_output = model(sample_image)
    _, vanilla_pred = torch.max(vanilla_output, 1)

    # TTA 예측
    tta_output = tta_model(sample_image) # 증강된 이미지들의 예측 결과를 평균
    _, tta_pred = torch.max(tta_output, 1)

print(f"Sample Image True Label: {class_names[sample_label]}")
print(f"Standard Prediction: {class_names[vanilla_pred.item()]}")
print(f"TTA Prediction: {class_names[tta_pred.item()]}")
```

```python
def knowledge_with_fallback(query: str) -> str:
    """벡터 DB 검색 후 결과가 부족하면 GPT로 fallback"""
    try:
        response = knowledge_chain.run(query)
        fallback_phrases = [
            "찾지 못했습니다",
            ...생략
        ]

        # 불충분한 답변이면 GPT로 직접 답변
        if any(phrase in response for phrase in fallback_phrases) or len(response.strip()) < 50:
            print(f"[fallback] knowledge_db 응답 부족 → direct_answer 재시도: {query}")
            # 농업 전문 프롬프트로 GPT 답변 생성
            agricultural_prompt = f"""[농업 전문 프롬프트 내용] """
            gpt_result = direct_chain.invoke({"query": agricultural_prompt})
            final_answer = gpt_result.content if hasattr(gpt_result, 'content') else
str(gpt_result)
            return response
```

```java
@Service
@RequiredArgsConstructor
public class ChatService {

    private final ChatRepository chatRepository;

    @Qualifier("pythonWebClient")
    private final WebClient pythonWebClient;

    /**
     * Python 챗봇 에이전트에게 질문하고 답변을 받습니다.
     * 항상 새로운 세션을 생성합니다.
     *
     * @param user      인증된 사용자 또는 null
     * @param question  클라이언트에서 받은 질문
     * @param chatId    세션 id 조회용 PK
     * @return 에이전트가 생성한 답변
     */
    @Transactional
    public ChatResponseDto askPythonAgent(User user, Long chatId, String question) {
        Long pythonSessionId = null;

        // --- 요청 처리: 클라이언트가 보낸 chatId(세션 ID)가 있는 경우 ---
        if (chatId != null) {
            Chat existingChat = chatRepository.findById(chatId)
                    .orElseThrow(() -> new IllegalArgumentException("존재하지 않는 채팅방입니다."));

            // (보안) 본인 채팅방이 맞는지 확인
            if (user != null && !existingChat.getUser().getUserId().equals(user.getUserId())) {
                throw new SecurityException("해당 채팅방에 접근할 권한이 없습니다.");
            }
            // FastAPI에 보낼 pythonSessionId를 DB에서 조회
            pythonSessionId = existingChat.getPythonSessionId();
        }

        // FastAPI 서버에 요청 (pythonSessionId가 null이면 FastAPI가 새 세션을 만듦)
        PythonChatDto.PythonChatRequest request = new PythonChatDto.PythonChatRequest(pythonSessionId, question);

        PythonChatDto.PythonChatResponse responseFromPython = pythonWebClient.post()
                .uri("/api/chat")
                .bodyValue(request)
                .retrieve()
                .bodyToMono(PythonChatDto.PythonChatResponse.class)
                .block();

        String answer = responseFromPython.getMessages().stream()
                .filter(msg -> "assistant".equalsIgnoreCase(msg.getRole()))
                .reduce((first, second) -> second)
                .map(PythonChatDto.PythonChatMessage::getQuery)
                .orElse("답변을 찾을 수 없습니다.");

        // --- 응답 처리: 첫 채팅이었던 경우 ---
        if (chatId == null && user != null) {
            Chat newChat = Chat.builder()
                    .user(user)
                    .pythonSessionId(responseFromPython.getId()) // FastAPI가 새로 만든 세션ID 저장
                    .build();
            Chat savedChat = chatRepository.save(newChat);
            chatId = savedChat.getChatId(); // 새로 생성된 DB의 chatId를 확정
        }

        // 최종적으로 클라이언트에게 보낼 응답 생성
        return ChatResponseDto.builder()
                .answer(answer)
                .question(question)
                .chatId(chatId) // 기존 chatId 또는 새로 생성된 chatId
                .build();
    }
```

## 2) BackEnd

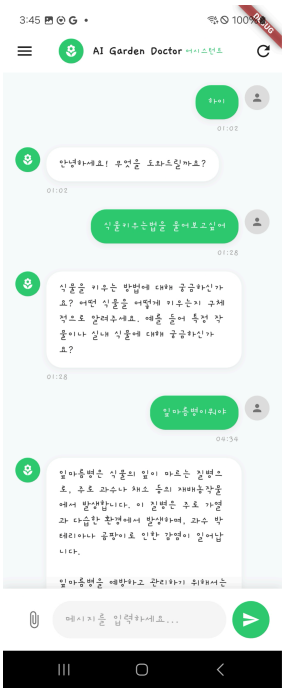
```java
/**
 * 매일 오전 9시에 오늘의 작업 알림 발송(각 userPlant)
 * - FCM 토큰이 유효하지 않는 사용자의 userPlant, 알림 수신 안 하는 userPlant 제외
 */
@Transactional
@Scheduled(cron = "0 0 9 * * *")
public void sendDailyTasksNotification() {
    Map<UserPlant, List<String>> tasksByUserPlant = new HashMap<>();

    // computeIfAbsent는 키(userPlant)에 대한 값(작업 목록)이 존재하면 해당 값 반환
    // k는 map에 존재하지 않는 키: 이 경우 새 리스트 생성 후 저장
    // 즉, 이미 존재하는 리스트 또는 새로 생성된 리스트에 add 수행
    List<UserPlant> needWateringToday = userPlantRepository.findUserPlantsNeedWateringToday();
    needWateringToday.forEach(up ->
            tasksByUserPlant.computeIfAbsent(up, k -> new ArrayList<>()).add("\uD83D\uDCA7물 주기"));

    List<UserPlant> needPruningToday = userPlantRepository.findUserPlantsNeedPruningToday();
    needPruningToday.forEach(up ->
            tasksByUserPlant.computeIfAbsent(up, k -> new ArrayList<>()).add("✂가지치기"));

    List<UserPlant> needFertilizingToday = userPlantRepository.findUserPlantsNeedFertilizingToday();
    needFertilizingToday.forEach(up ->
            tasksByUserPlant.computeIfAbsent(up, k -> new ArrayList<>()).add("\uD83D\uDC8A영양제 주기"));

    tasksByUserPlant.forEach((key, value) -> {
        User user = key.getUser();
        Long userPlantId = key.getUserPlantId();
        String plantName = key.getPlantName();
        String plantNickname = key.getPlantNickname();
        String tasks = String.join(", ", value);
        log.info("Daily tasks for UserPlant ID({}): {}", userPlantId, tasks);
        String message = String.format("%s(%s)의 오늘 해야 할 작업: %s. 잊지 말고 꼭 챙겨주세요!",
                plantNickname, plantName, tasks);
        notificationService.createAndSendNotification(user, "\uD83E\uDEB4오늘의 작업 알림", message);
    });
}

/**
 * 매일 오후 5시에 미완료 작업 알림 발송(각 userPlant)
 * - FCM 토큰이 유효하지 않는 사용자의 userPlant, 알림 수신 안 하는 userPlant 제외
 */
@Transactional
@Scheduled(cron = "0 0 17 * * *")
public void notifyIncompleteTasks() {
    Map<UserPlant, List<String>> incompleteTasksByUserPlant = new HashMap<>();

    List<UserPlant> incompleteWateringToday = userPlantRepository.findUserPlantsIncompleteWateringToday();
    incompleteWateringToday.forEach(up ->
            incompleteTasksByUserPlant.computeIfAbsent(up, k -> new ArrayList<>()).add("\uD83D\uDCA7물 주기"));

    List<UserPlant> incompletePruningToday = userPlantRepository.findUserPlantsIncompletePruningToday();
    incompletePruningToday.forEach(up ->
            incompleteTasksByUserPlant.computeIfAbsent(up, k -> new ArrayList<>()).add("✂가지치기"));

    List<UserPlant> incompleteFertilizingToday = userPlantRepository.findUserPlantsIncompleteFertilizingToday();
    incompleteFertilizingToday.forEach(up ->
            incompleteTasksByUserPlant.computeIfAbsent(up, k -> new ArrayList<>()).add("\uD83D\uDC8A영양제 주기"));

    incompleteTasksByUserPlant.forEach((key, value) -> {
        User user = key.getUser();
        Long userPlantId = key.getUserPlantId();
        String plantName = key.getPlantName();
        String plantNickname = key.getPlantNickname();
        String tasks = String.join(", ", value);
        log.info("Incomplete tasks for UserPlant ID({}): {}", userPlantId, tasks);
        String message = String.format("%s(%s)의 아직 완료하지 않은 작업: %s. 잊지 말고 꼭 챙겨주세요!",
                plantNickname, plantName, tasks);
        notificationService.createAndSendNotification(user, "⚠오늘의 미완료 작업 알림", message);
    });
}
```

## 4.3 UI/UX 화면 캡처

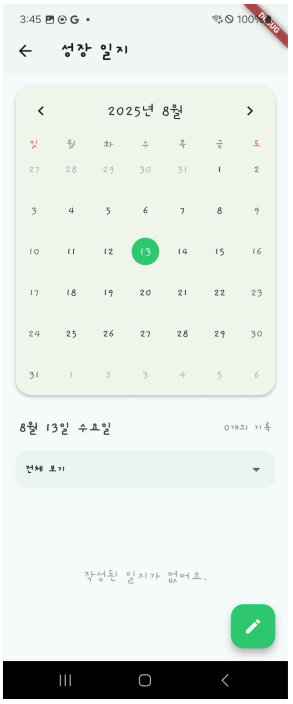
- 앱 화면

<로그인 화면>

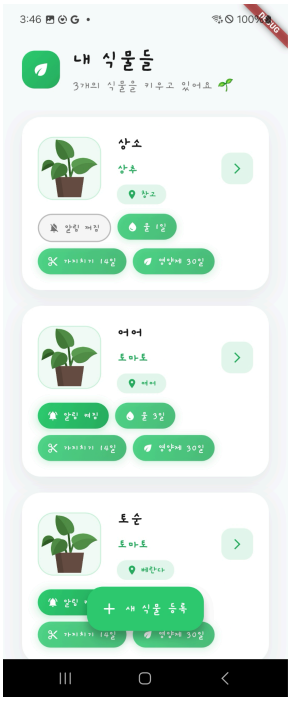
<AI 작물 진단 화면>



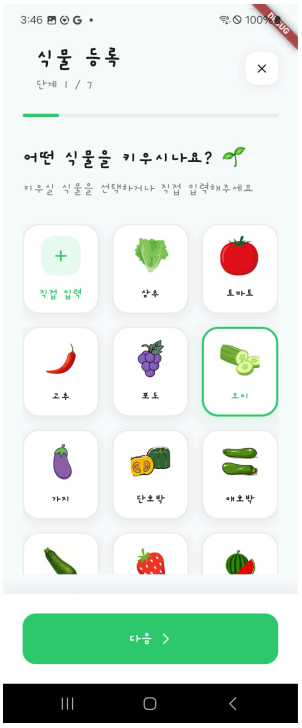

<AI 챗봇 대화 화면>



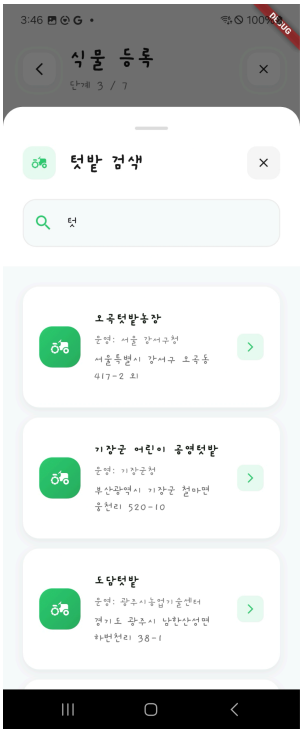

<작물 성장일지 화면>



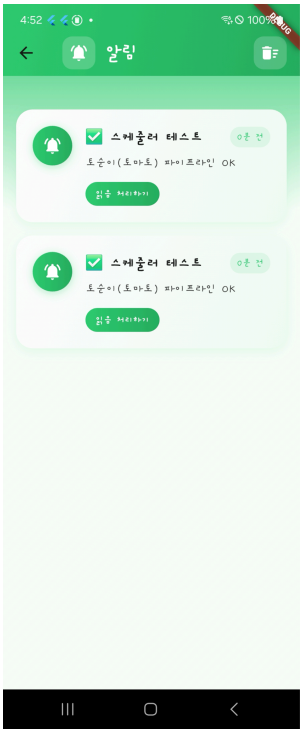

<내 작물 관리 화면>




<식물 등록 화면>

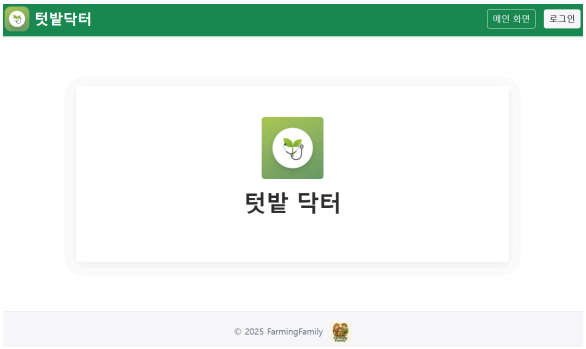



<주변 주말농장 화면>

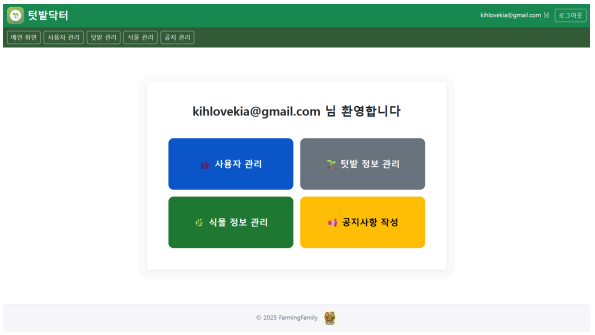



<알림창>

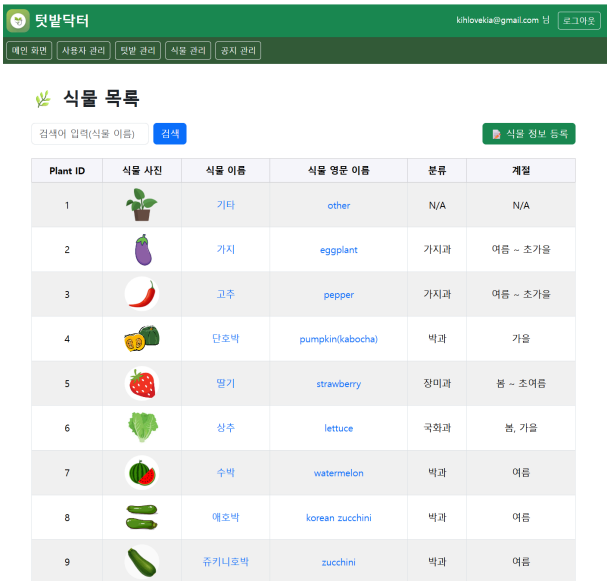
- **관리자 페이지 화면(일부)**

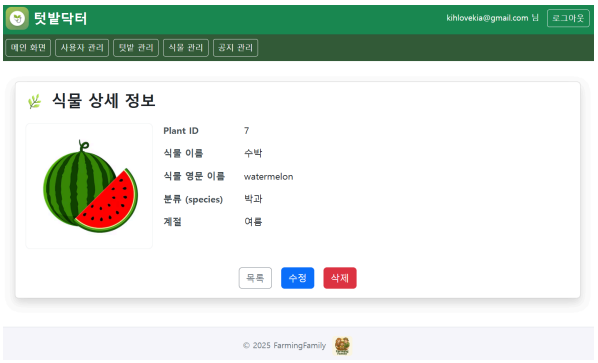



<메인화면>

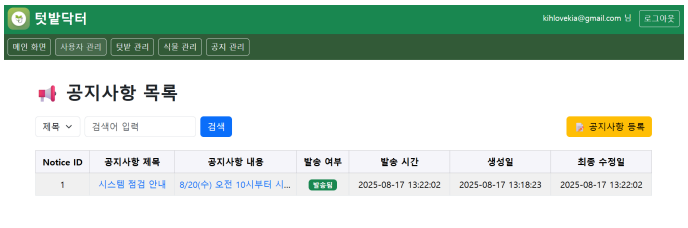


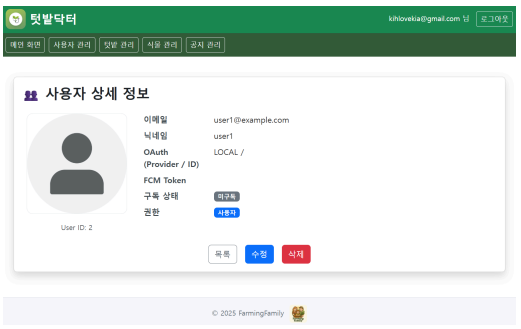

<관리자 로그인>




<식물 목록, 식물 상세 정보 페이지>




<공지사항 목록, 사용자 상세 정보 페이지>

## 4.4 이슈관리

프로젝트를 진행하면서 발생한 일정 변경, 사양 변경, 인력 변경, 속성 변경 등 협의가 필요한 모 든 사항은 이슈로 간주한다. 프로젝트 관리자는 발생한 모든 이슈를 기록하여 관리하고 적절한 일정에 따라 이슈가 해결 되는지 여부를 주기적으로 관리한다.

- 기술: AI 모델 성능, API 연동 문제 등 기술적 이슈
- 일정: 개발 지연, 일정 조정이 필요한 사항
- 데이터: 데이터 품질, 수집 관련 이슈
- 외부: 외부 API 변경, 정책 변경 등 외부 요인

| 유형 | 내용 | 주기/방법 | 산출물 | 담당자 |
|---|---|---|---|---|
| **주간회의 (내부)** | 개발 진행 현황·이슈·형상관리 점검,<br>다음 주 계획 수립 | 매주 금요일/회의 | 주간보고서 제출 및 피드백 | 남은지 |
| **일일 스탠드업** | 전일 작업 완료·금일 계획 공유, 칸반 보드 업데이트 | 매일 notion에 작성 | Notion 칸반보드 | 전체 |
| **기술 검토 회의** | 아키텍처 검토, 기술적 이슈 해결 | 필요 시 수시로 연락 | 카톡·Notion | 개발팀 |
| **멘토링 세션** | 진행 상황 보고, 기술 조언·방향 점검 | 시간 조정/온라인 or 대면 | 회의록 | 전체 |
| **주간보고(외부)** | 강사님께 현황·미진 항목 보고 | 매주 시간 조정 후 회의 | 주간보고서 | 전체 |
| **마일스톤 보고** | 주요 개발 단계 완료 보고 | 단계 완료 시 발표 | 마일스톤 보고서 | 개발팀 |

## 4.5 설치/운영 매뉴얼

산출물_AI 3팀_파밍패밀리

1) API 명세서
2) 기능 명세서
3) 요구사항 명세서

## 4.6 참고문헌


**1) 정부 부처·공공기관 보도자료·정책·통계**

- 농림축산식품부
    - 2024년 농식품부 주요업무 추진계획(보도자료, 2024.03.04)
    - 도시농업현황조사(과학기술정책과, 연도 미상)
    - 도시농업 참여율 조사 통계(2021)
    - 도시농업 미참여 사유 조사(N=1,000, 2020)
    - 도시농업 관련 예산 현황(2017–2020)
    - 도시농업 경제적 가치 평가 연구(2017)
    - 2025년 업무계획(농업진흥구역 관련 규제 완화)
    - 2025년 농림축산식품부 예산안(총 18조 7,496억원)
- 농촌진흥청
    - 도시농업 참여자 통계(2010: 15만명 → 2019: 241만명)
    - 스마트 병해충 진단 시스템 관련 성과치(피해율 연간 30~40% 감소, 수확량 20~30% 증가)
    - 농업기술 상담예약 시스템(전화상담·현장방문 예약 지연 관련 운영 기준)
    - NCPMS 온라인 상담 통계(병해충 온라인 의뢰의 79%가 재배관리/생리장해/병해 관련)
    - 샤인머스켓 바이러스·바이로이드 감염 피해 통계(수확량 69% 감소, 1ha당 2,070만원 손실)
    - 농사로(NongSaRo) Open API 및 전문가 칼럼(2021)
- 한국농촌경제연구원(KREI)
    - 도시농업 비전 보고서(2010, 2022): 도시농업 이탈 원인 분석(병해충/재배 실패)
- 통계청
    - 대구·경북 청년 농가 수 변화(2012년 57,000여 명 → 10년 후 25,000여 명)
- 경기도농업기술원
    - ‘사이버 식물병원’ 3개년 분석(2011–2013, 901건: 채소 47.1%, 화훼 24.3%, 의뢰사유 생리장해 31.7%, 병해 28.6%)
- 서울특별시
    - 자치구 도시농업지원센터 운영점검(2021): 장기 병해·생리장해에 따른 중도 포기·보조금 환수

        - 서울시 농업기술센터 공무직 채용공고(병해충·종합분석실 인력·전문가 비율 관련 서술 근거)
        - 공유텃밭 경쟁률(6:1), 시민 인식/참여 의향(아파트 공유텃밭 참여의향 89.2% 등)

2) 보도자료·언론·외부 리포트

- KBS 보도(의성군 청년 귀농·귀촌 사례, 스마트팜 조성 사업 교육 포기·창업 비율)
- 농민신문(병해충 발생 집계·전문인력 부족, SOP 반영 한계 관련 보도)

3) 공공데이터·데이터셋·API

- AI Hub
    - 시설작물 질병 진단 이미지 데이터셋
    - 노지 작물 질병 진단 이미지(고추: 정상/탄저병/흰가루병 등 라벨 기준 활용)
- 농림축산식품 공공데이터 포털
    - 텃밭 분양 상세정보 데이터셋
- 경기데이터드림
    - 텃밭 분양 정보 데이터셋
- 농촌진흥청/농림수산식품교육문화정보원/경기도 등
    - 농업 정보 API, 공공데이터 API
- 농사로(NongSaRo) Open API
    - 작물 재배법·병해충 정보 문서 메타데이터·PDF 수집 및 RAG 근거 구축용

4) 벤치마킹·비교 근거(서비스 지표·레퍼런스)

- 농촌진흥청 병해충 영상진단 AI 평균 정확도 약 95% 수준
- NH오늘농사 앱
    - 월간 재방문율 50% 이상(최대 62% 수준)
    - 유사 서비스 출시 초/평균 DAU 500~2,500명 근거로 DAU 5,000명 목표 설정
- 농진청 스마트 병해충 진단 시스템
    - 피해율 30~40% 감소, 수확량 20~30% 증가 실증 근거

6) 정책·제도 변화 및 사업환경 근거

- 농업진흥구역 주말농장 운영 규제 완화(2025년): 도시농업 참여 기회 확대, 위치기반 공급 확대
- 농림축산식품부 3대 전환 정책(디지털 전환, 세대전환, 농촌공간 전환): 텃밭닥터의 디지털 전환·도시농업 연계 근거